COMPUTER ENGINEERING
QUEZON CITY

Final Project: Laboratory Equipment Borrowing and Inventory Management System

Course Code: CPE 009B **Course Title: Object Oriented Programming 2**

PROJECT INFORMATION

Item	Details
Project Title	Laboratory Equipment Borrowing and Inventory Management System
Section / Group No.	CPE21S4 / Group 3
Instructor	Engr. Jimlord M. Quejado
Date Submitted	Nov 3, 2026

Group Members

#	Name (Surname, First Name M.I.)	Student No.	Role / Main Contribution
1	Acelajado, John Vince D.	2516965	Front End, Styling, Github, Documentator, PPT Developer
2	Bal, Kevin Ranz J.	2513850	Front-end, Back-end, Styling, Documentator, Github

Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Develop a solution to an industry-related problem (the group's chosen real-world problem) utilizing object-oriented analysis and design.
- Integrate the use of the object-oriented programming language, Python, with the design of a computer program by creating and using multiple classes.
- Create and present a program that solves a real-world problem using best coding practices in Python and implements Object-Oriented design principles such as encapsulation and composition

Objectives

General Objective: To develop a PyQt6 desktop application that will take a laboratory borrowing slip through a digital workflow which the Student submit a student slip request, the request is saved at the ITSO CSV Log, the request is approved or denied by the Professor, and the Laboratory Staff release and retrieve the materials and maintain an accurate inventory.

Specific Objectives:

1. To create role-based logins and different dashboards for students, Itso, professors and laboratory staff, and have a borrow-slip workflow (request, saved at csv log, approve or reject, update the inventory csv based what material borrow, release, return).SC
2. To validate all user entry (login credentials, student details about ID and name, quantities, required fields, equipment availability), and provide feedback to the student user via message and dialog boxes.
3. To store and load all of the students borrow slips requested in a CSV file to retain data records of all the requests. Additionally, having a CSV file for inventory so the data will retain even the application was closed
4. To apply the object-oriented principles of encapsulation, abstraction, inheritance, and polymorphism in creating the classes which included the user class, equipment class, slip, inventory management, data handling and also the window classes.
5. To package the application as a Windows executable (.exe) that will run without a Python installed.

1. INTRODUCTION

1.1 Background of the Project

In borrowing laboratory equipment in a school, the current process starts when a student gets a borrowing slip from the laboratory staff. The student fills out the required information and asks the professor to sign the slip for approval. Once the professor has signed the slip, the student will give the approved slip and their ID to the laboratory staff. The staff checks the approval of the professor based on the signature, then release the requested materials, and records its release and return.

This method is inefficient. It is time-consuming for the student to fill out the form and wait for the professor's approval. The student may also find out only at the laboratory that the needed item is already borrowed or unavailable. Filling out the slip by hand can also lead to erasures, missing quantities, missing signatures, and incomplete information and also ITSO does not have any organized records about the slip. This project addresses these problems by replacing the paper slip with a desktop application, which makes the process more organized, optimal and keeps all records saved in one place.

1.2 Statement of the Problem.

The manual borrowing slip system has the drawback of slowing the process of requesting and approval of laboratory equipment for students and it is difficult for the school to keep track of equipment availability and borrower slip records, who has borrowed what and ensure equipment is properly released and returned.

1. Requesting and approving a borrower slip takes a lot of time because the student needs to do it manually because the student needs filling out the borrower slip, getting an approval signature of the prof and to line up to the staff laboratory area to get the materials.
2. The handwritten slips may involve some erasures, missing material details and also wrong quantities, and the student may discovers at the equipment area where the item is already unavailable.
3. There is no systematic record of each request, so it is not easy to find out who has borrowed which equipment, and to know what status of the borrowed equipment.

1.3 Target Users

User	Description	What They Do in the Application
Students	Requester	They fill out the materials and equipment they need for their laboratory equipment
Professors	Reviewer and Approver	They review and approve the request forms of the students that require laboratory equipment
Staff in the Laboratory	Checking the approval of the professor, Inventory keeper, Releaser of material	Providing the requested laboratory equipment of the student/s based on what the professors approve and they are the one who check and update the availability of the materials.
ITSO	Records keeper	They will be the one who keeps every record of the slip via CSV log.

1.4 Significance of the Project

This project benefits the members of TIP especially the Students who get a faster and more clearer way to request the laboratory equipment, check availability and the status of the request slip. Professors can review all the slips requested by the students and approve them anytime. ITSO will keep the records of every slip to make it organized, Lastly laboratory staff update the inventory management and review the slip that the Professor approved and release the laboratory item in the slip. This project supports the SDG9(Industry, Innovation, and Infrastructure) by improving the manual borrowing slip process into a digital process which improves how the process of borrowing equipment is more convenient and this makes the workflow smooth and has a reliable record.

2. SCOPE AND DELIMITATIONS

2.1 Scope

#	Feature	Description	Window / Page
1	Login System	Role dropdown (Student, Professor, Laboratory Staff, ITSO Maintenance), username and password fields, a "Show password" checkbox, a "Clear" button to reset the inputs, and finally a "Login" button that will open the dashboard of the selected role. If the user types in an incorrect login and password, an error message will display and the window will shake.	Login Window

2	Student Dashboard	Form for Student ID, Student Name, Year Level, Course & Section, Academic Program, Date Needed and Assigned Professor (the professor list varies depending upon the program). It also contains a tracking table of the students' requests (Request ID, Items, Status, Lab Ticket ID, Update), a list of any student's professor announcements, a search feature for materials availability, "Add Item" and "Clear Items" buttons for the materials list, a "Submit Borrowing Request" button, and a “Return to Staff” button for each released request.	Student Dashboard Window
3	Professor Dashboard	Table of requests that are coming in (Student ID, Name, Course & Section, Date Needed, Requested Items, Status, Time Submitted). The professor clicks on a row and then clicks to "Approve Request" or "Decline Request". If declines, must provide a comment, otherwise a pop-up asks for a reason. The professor also has the ability to post an announcement which will be seen on the student dashboard.	Professor Dashboard Window
4	ITSO Dashboard and Borrow Slip Records	Table of all slips saved automatically in the CSV log (Request ID, Student ID, Name, Course & Sec, Date Needed, Items, System Status, Lab ID, Time Submitted, Time Returned). Sync / Refresh CSV Log: Reloads the CSV Log. Generate Backup Folder: Generates a folder named ITSO Backup Folder which contains a copy of the CSV Log. Delete Backup Folder: Deletes a user-selected folder.	ITSO Dashboard Window
5	Laboratory Staff Dashboard and Inventory Management	The function "Update / Add Stock" is used to add quantity to an existing material or to create a new material in the Current On-Hand Stock Inventory Ledger (Material/Hardware Model, Units Available). The Verified Request Release table contains Requests with "Release Items" (asks for Lab Ticket ID and checks stock) and "Confirm Return" buttons.	Laboratory Staff Dashboard Window

2.2 Delimitations

- The application will not use database (MySQL, MySQLite3, & SQLite)
- The application is limited to TIP Students, TIP Professors and TIP Laboratory Staff only
- The account switching feature will demonstrate the four user roles but will not provide a complete authentication and user registration.
- The application will not physically collect or verify the student’s ID, The laboratory staff will remain responsible for checking and holding the physical ID.

- The application will run on a single desktop computer only and will not connect or synchronize the information/data that it receives from the app when executing it across multiple computers.
- The application will not provide automated email or SMS notification. The notifications like announcement from the Professor Dashboard will be displayed only in the application.
- The application will not automatically detect the damage, missing, or incomplete materials. The Laboratory Staff will remain responsible to inspect, return items and enter relevant information.
- The system will not use barcode or RFID to identify students or laboratory materials.
- The system will not support simultaneous borrowing requests from multiple users on separate devices, as it is designed to demonstrate the workflow within a single running application.
- The Project will focus on developing and demonstrating the prototype rather than applying it to a full operation of the system across several laboratories.

2.3 Minimum Technical Requirements

Done	Requirement
[/]	A desktop GUI built with PyQt6
[/]	At least four classes of your own, organized in at least three Python files (modules)
[/]	Encapsulation, abstraction, inheritance, and polymorphism, each applied at least once
[/]	Input validation with clear feedback to the user
[/]	Data saved to and loaded from a file (e.g., .txt, .csv, or .json)
[/]	At least two windows or pages, with navigation between them
[/]	A menu bar or toolbar, and at least one dialog
[/]	Consistent styling using Qt Style Sheets (QSS)
[/]	The application packaged as a working Windows .exe

3. EXPECTED OUTPUT

3.1 System Overview

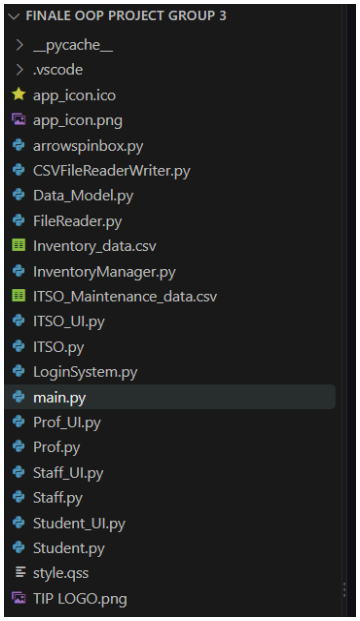
The first screen the user will encounter is the Login Window in which they select a role and type in a username and password. A student opens the Student Dashboard, searches material availability, adds a list of materials and submits a borrowing request, saving it to the ITSO CSV log in the state "Pending Approval". A professor opens the Professor Dashboard and either approves or denies the request (must include reason for denial) and may write announcements for students. Laboratory personnel access the Staff Dashboard and handle the stock ledger. They also release the materials through the dashboard verified request and they confirm if the materials are returned. ITSO Maintenance accesses the ITSO Dashboard to see the status of each slip, Lab ID and submitted and returned times, and can create and delete a backup folder of the CSV log. Each dashboard includes an “Active Account” tag and a “Logout / Switch System Role” button which will bring the user back to the Login Window.

3.2 Tools and Technologies

Tool	Version	Purpose
Python	3.12	Programming language
PyQt6	6.11.0	Graphical user interface
Visual Studio Code	1.94.2	Code editor
PyInstaller	6.22.3	Packaging as a Windows .exe
draw.io	Web version	Drawing the class diagram

3.3 Project File Structure

```
[FINALE OOP PROJECT GROUP 3]/
├── main.py                  # Entry point where loads data, login and open windows
├── LoginSystem.py           # login window which has role, credentials and show password
├── Student.py               # student where it submit slip, search stock, return
├── Student_UI.py            # student layout where it has form, track, and item tables
├── Prof.py                  # prof which has approve, decline, and announcements
├── Prof_UI.py               # professor layout which contains request table and comment box
├── Staff.py                 # staff which contain release, confirm return, and add stock
├── Staff_UI.py              # staff layout which contains stock ledger and release table
├── ITSO.py                  # ITSO which load the CSV log and make/delete backup folder
├── ITSO_UI.py               # ITSO layout which has log table, and can sync/backup buttons
├── InventoryManager.py      # inventory contains add, borrow, return and it will save to
CSV
├── Data_Model.py            # contains data classes of slip, stock, announcement
├── FileReader.py            # where parent class FileReaderWriter( read and write)
├── CSVFileReaderWriter.py   # child class which contains reads and appends the CSV rows
├── arrowspinbox.py          # spin box contains the custom draw up and down arrows
├── style.qss                # the theme is yellow, gray, and black like TIP logo
├── app_icon.ico / app_icon.png / TIP_LOGO.png  #icons and logo of the project
├── Inventory_data.csv       # it saved stock counts
└── ITSO_maintenance_data.csv# it saved the borrow slips
```



3.4 Class Diagram

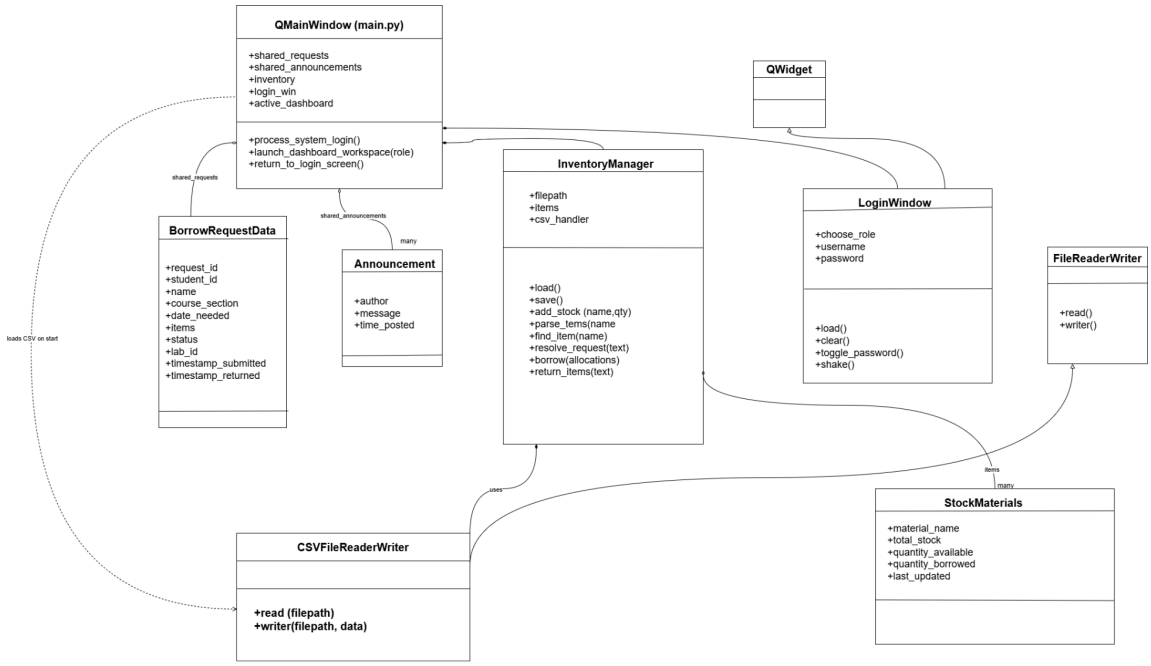


Figure 1. Class diagram of the controller, data and file-handling classes

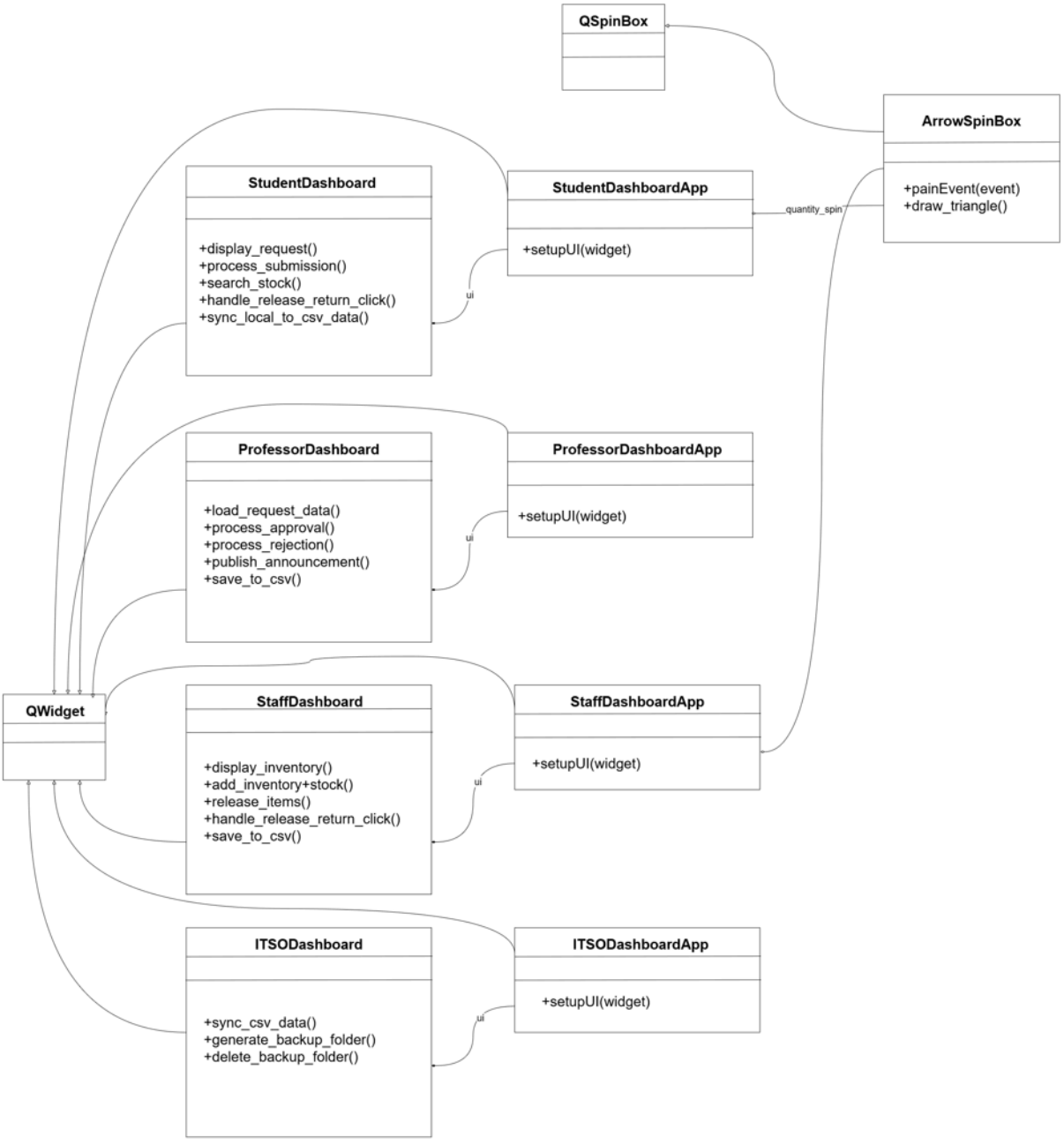


Figure 2. Class diagram of the dashboard (window) classes

3.5 Expected Output

Deliverable	Description
Working application	Source code that runs with python main.py (15 python modules, style qss, icons)
Windows executable	https://drive.google.com/file/d/113gFCSS5H0xnyuHubIvTLxRzDqh4TOov/view?usp=sharing
Project report	https://docs.google.com/document/d/1EmeXt2JzVU8eLoRnAY3w9plpCZKzZd2cAB-vthR_dvE/edit?tab=t.0
Presentation	https://www.canva.com/design/DAHZw8x6UKc/fXf9yjSvyq016BJun6TC5w/edit

4. APPLICATION OF OBJECT-ORIENTED PROGRAMMING PRINCIPLES

4.1 Classes and Objects

Class	File	Inherits From	Responsibility
LoginWindow	LoginSystempy	QWidget	Stores one record and validates its own data
CSVFileReaderWritter	CSVFileReaderWriter.py	FileReaderWritter	Converts those inputted data of the student in the borrowing slip into a CSV File.
FileReaderWriter	FileReaderWritter	None	Parent Class of the CSVFileReaderWriter, that contains the base layout for reading and writing files.
BorrowRequestData	Data_Model.py	None	Contains all of the data that the student will input in the borrowing slip, including the Laboratory ID, and timestamp of submission and return of the material.
Announcement	Data_Model.py	None	To track each announcement message that will publish of the Instructor
StudentDashboard	Student.py	QWidget	Shows the Dashboard GUI for the Student/s
StudentDashboardApp	Student_UI.py	QWidget	Handles all of the layout fields combining the logics of the Student.py
ProfessorDashboard	Prof.py	QWidget	Shows the Dashboard GUI for the Professor/s

ProfessorDashboardApp	Prof_UI.py	QWidget	Handles all of the layout fields combining the logics of the Prof.py
StaffDashboard	Staff.py	QWidget	Shows the Dashboard GUI for the Laboratory Staff
StaffDashboardApp	Staff_UI.py	QWidget	Handles all of the layout fields combining the logics of the Staff.py
ITSODashboard	ITSO.py	QWidget	Shows the Dashboard GUI for the ITSO
ITSODashboardApp	ITSO_UI.py	QWidget	Handles all of the layout fields combining the logics of the ITSO.py
ArrowSpinBox	arrowspinbox.py	QSpinBox	A number box for the quantity that draws its own up and down arrows so they can be seen with the QSS theme.
InventoryManager	InventoryManager.py	None	Keeps the stock list, loads and saves Inventory_data.csv, checks if there is enough stock, and updates stock when items are added, borrowed or returned.
LabPortalApp	main.py	None	The main controller. It loads the saved data, shows the login window, checks the login, opens the dashboard of the chosen role, and handles the menu bar and logout.
StockMaterials	Data_Model.py	None	Stores one line of the inventory: name, total stock, available units, borrowed units, and last updated time.

4.2 Encapsulation

Where applied: [Class name, method or property name]

```
def add_stock(self, name, qty):
       # adds  to  an existing item (same name, any  capitalisation) or  creates a brand new
one
       stamp = self.now()
       for s in self.items:
           if s.material_name.lower() == name.lower():
               s.total_stock += qty
               s.quantity_available +=  qty
```

```
                s.last_updated  =  stamp
                self.save()
                return s
        new_item = StockMaterials(name , qty , last_updated  =  stamp)
        self.items.append(new_item)
        self.save()
```

Explanation: The stock data and the way it is saved are kept inside the InventoryManager class. The dashboards do not edit the stock numbers or the CSV file by themselves. They only call methods like add_stock(), check_request(), borrow() and return_items(). Each method checks the data first.

4.3 Abstraction

Where applied: CSVFileReaderWriter class, read() and write() methods

```
class CSVFileReaderWriter(FileReaderWriter):
   def read(self, filepath):
       with open(filepath, newline='') as  csvfile:
           data = csv.reader(csvfile, delimiter=',', quotechar='|')
           return list(data)
```

Explanation: When the dashboard wants to load data, it calls the csv_handler.read(filepath). The dashboard does not need to know how python opens a file, handles delimiters, or parses strings. The dashboard does not need to know the exact steps of how python opens a file, uses delimiters, or processes lists. It hides those unorganized details inside this method.

4.4 Inheritance

Where applied: CSVFileReaderWriter class (Child) inherits FileReaderWriter class (Parent)

```
# CHILD CLASS
from FileReader import FileReaderWriter
import csv


   # this  file and function  is used for converting  the  borrowing  slip  of  the  student
   # from  the  student  dashboard  into csv  file and  received  to the itso dashboard


   # opens  a csv file  and  returns  all  lines as a list
class CSVFileReaderWriter(FileReaderWriter):
   def read(self, filepath):
       with open(filepath, newline='') as  csvfile:
```

```
            data = csv.reader(csvfile, delimiter=',', quotechar='|')
            return list(data)


    # opens  a csv  file and  adds a  new  line of  data to  the  end of  it
    def write(self, filepath, data):
    # opens the  file to  add new  student  lines without  deleting  the  old  ones
        with open(filepath, 'a', newline  = '') as csvfile:
            writer = csv.writer(csvfile,  delimiter=',', quotechar='|',
quoting=csv.QUOTE_MINIMAL)
            writer.writerow(data)
```

Explanation: The FileReaderWriter is the parent that acts as a blueprint for handling files. CSVFileReaderWriter is the child class, it inherits from the FileReaderWriter, meaning it automatically copies that structure and it then adds a specific method for handling the CSV data, this saves time because it doesn't need to write the layout from scratch.

4.5 Polymorphism

Where applied: CSVFileReaderWriter.read() overrides FileReaderWriter.read()

```
# create  our parent class
class FileReaderWriter():
    # base  layout for  reading  files  that returns an empty  list  if  not  changed
    def read(self):
        print("This is the default read method")
    # returns an  empty  list as a safety  backup to prevent the  program from crashing if no
data is  found
        return[]
    # base  layout for  writing data into files
    def write(self):
        print("This is the default write method")
```

Explanation: It allows a method to act differently depending on the specific object that calls it. The parent class FileReaderWriter has a default read() method that just prints a message and returns an empty list. The child class CSVFileReaderWriter overrides this method by using the exact same name but replacing the inside logic with instructions to analyze spreadsheet files. When the code calls read(), python runs the specific child version instead of the default parent layout.

4.6 Composition and Modularization

Where applied: LabPortalApp contains LoginWindow, and InventoryManager. InventoryManager contains CSVFileReaderWriter and StockMaterials. Each dashboard contains its own UI class.

```
# main.py, inside LabPortalApp.__init__
self.inventory = InventoryManager()
self.shared_inventory = self.inventory.items
self.login_win = LoginWindow()

# InventoryManager.py
self.csv_handler = CSVFileReaderWriter()
self.items = []

# Student.py, inside StudentDashboard.__init__
self.ui = StudentDashboardApp()
self.ui.setupUi(self)
```

Explanation: The main program does not do everything itself. LabPortalApp holds the login window and the inventory manager, and it gives the shared lists to the dashboards. The inventory manager holds a CSV handler and the stock items. Each dashboard holds its own layout class (self.ui) and keeps its logic in a separate file, such as Student_UI.py for the layout and Student.py for the logic. Splitting the code into 15 files made it easier for the group to work at the same time and to find errors. If a button looked wrong, we checked the UI file. If a rule was wrong, we checked the logic file. It also let us test each dashboard alone.

4.7 GUI Concepts Applied

Course Topic	Used? (Yes / No)	Where in the Project
Signals and slots	Yes	clicked.connect on buttons in all dashboards; textChanged in StudentDashboard.search_stock; currentTextChanged in StudentDashboard.update_professor_list; toggled in LoginWindow.toggle_password
Event binding and callbacks	Yes	self.sender() with the row_index property in handle_release_click and handle_release_return_click; ArrowSpinBox.paintEvent override
Forms and input validation	Yes	StudentDashboard.process_submission and add_item_to_table; ProfessorDashboard.process_rejection; StaffDashboard.add_inventory_stock; InventoryManager.check_request
Classes and modularization in GUI	Yes	One UI class and one logic class per dashboard (Student_UI.py / Student.py and so on)

File I/O integration	Yes	CSVFileReaderWriter; main.py (loading on startup); InventoryManager.save; ITSO.generate_backup_folder
Dialogs, menus, and toolbars	Yes	QMessageBox, QInputDialog (Lab Ticket ID), QFileDialog (backup folder); no menu bar or toolbar yet
Multi-window and navigation	Yes	main.py: process_system_login, launch_dashboard_workspace, return_to_login_screen
Styling GUIs (QSS)	Yes	style.qss loaded in main.py; objectName selectors such as pageTitle, sectionLabel, addItemBtn
Packaging as an .exe	Yes	It was done using pyinstaller process

5. RESULTS AND DISCUSSION

5.1 Application Screens

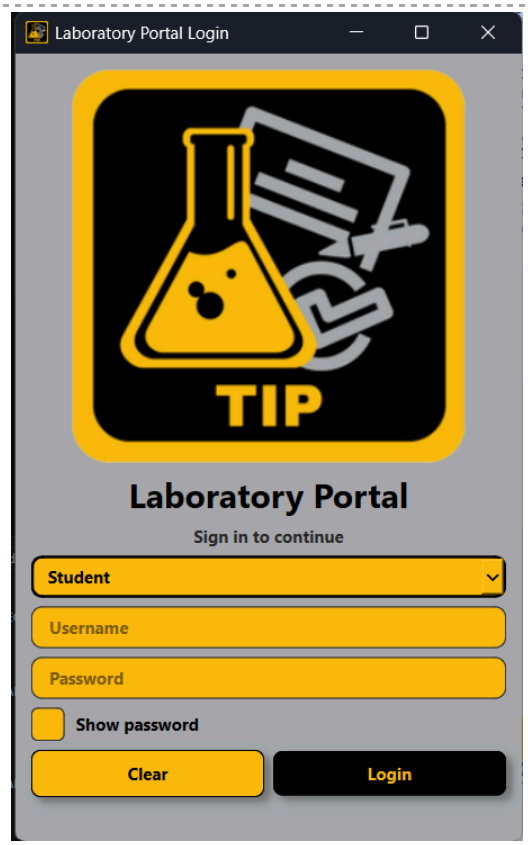


Figure 1. Login Window

When the program runs, this is the first one will appear, it depends on the user/s of which role they would login, it needs to make sure that the role was aligned to its username and password, and after logging in, it directs to a specific dashboard which the user picked earlier, and also have a show and clear password toggle for the user/s.

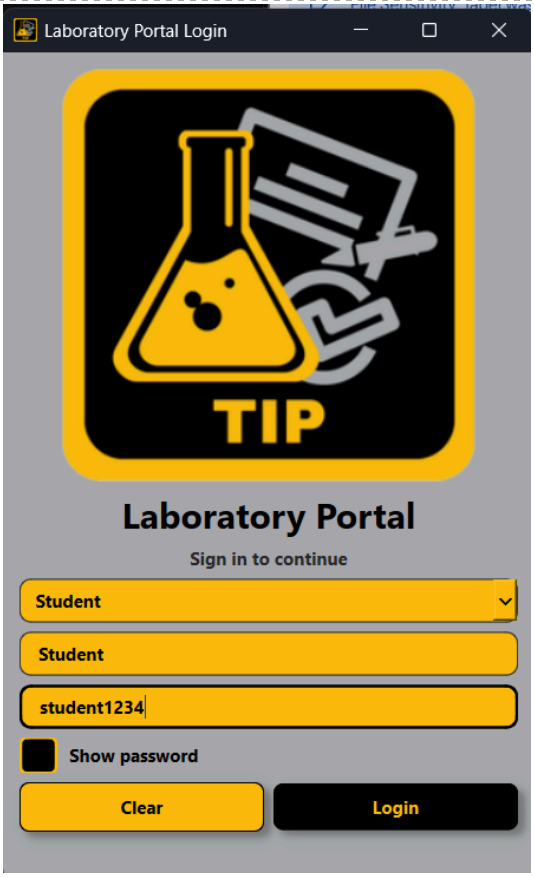


Figure 2. Student Login:

When the user Login as a Student, it should be matched the role, username, and password as a Student and it will be directed to the Student Dashboard

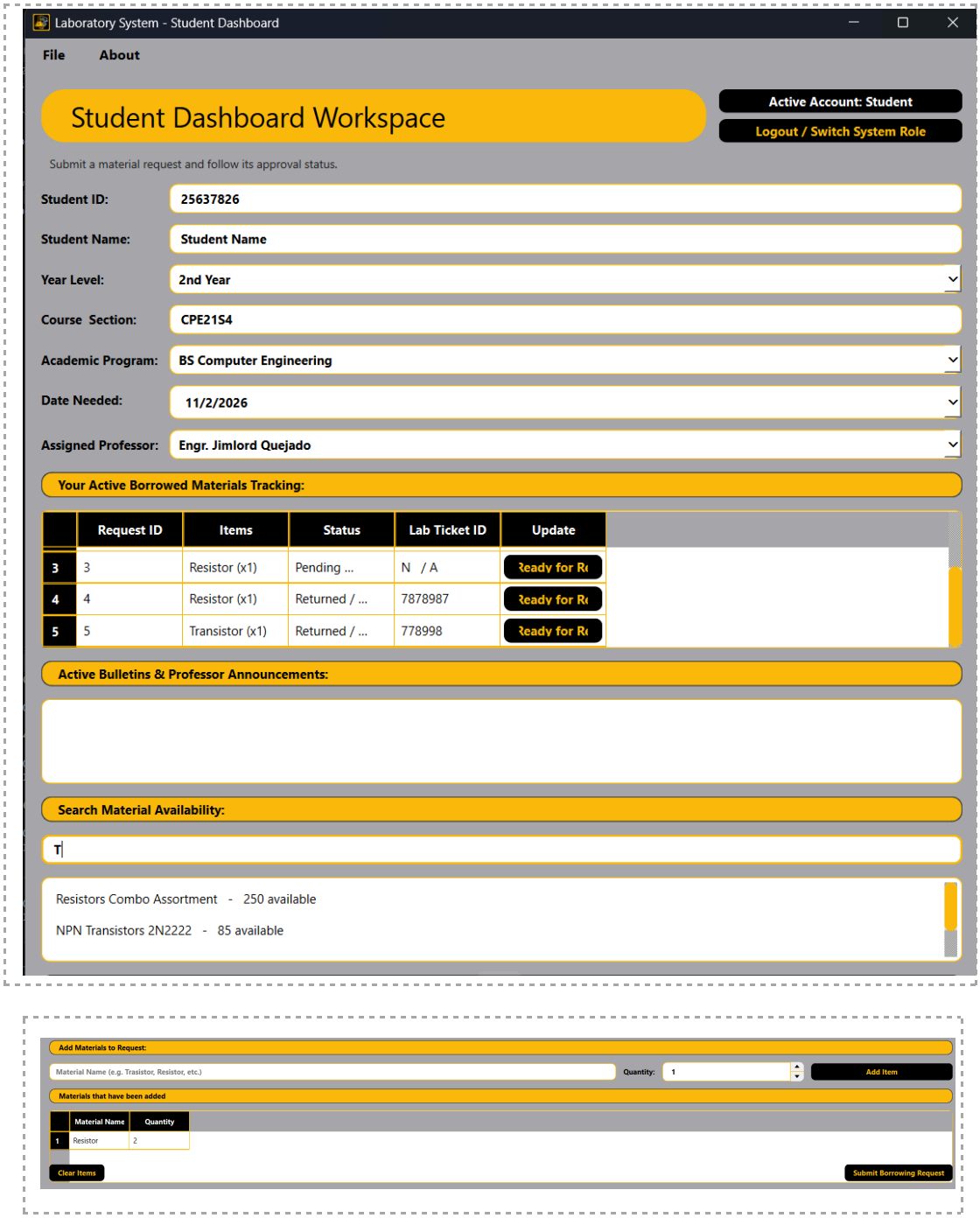


Figure 3. Student Dashboard:

The user/s must fill up all the requirements inside the Student Dashboard, including the material that is needed and there's also adding a quantity of that specific material before clicking "Add Item".

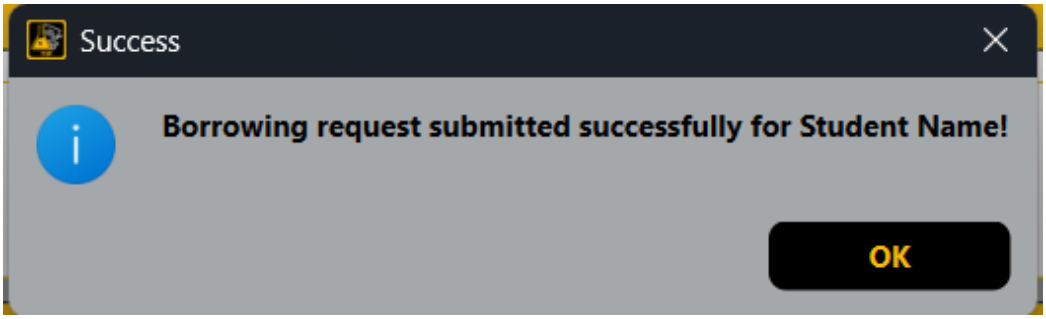


Figure 4. Student Dashboard:

If the user is already on the material its needed, it should click the submit button and there's a notification that would appear after submitting it

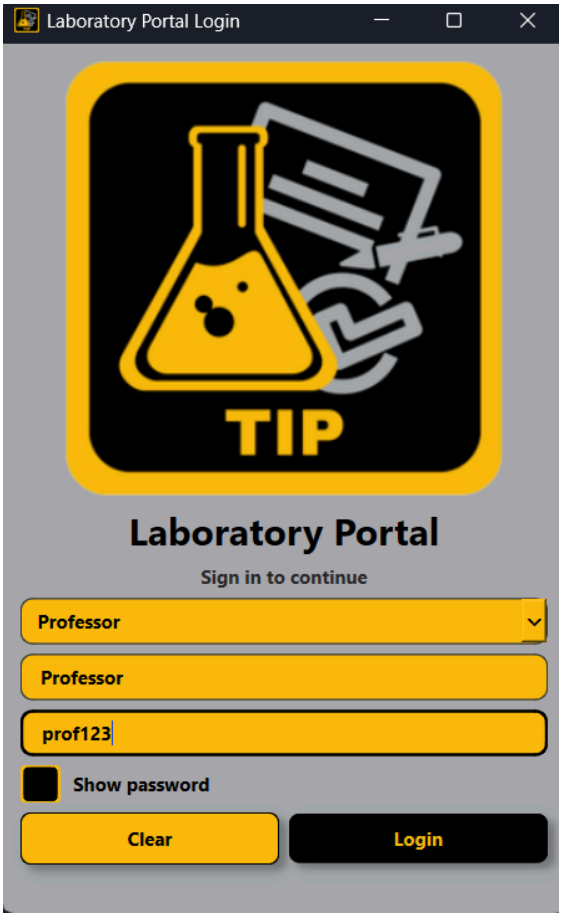


Figure 4. Professor Login:

When the user Login as a Professor, it should be matched the role, username, and password as a Professor and it will be directed to the Professor Dashboard

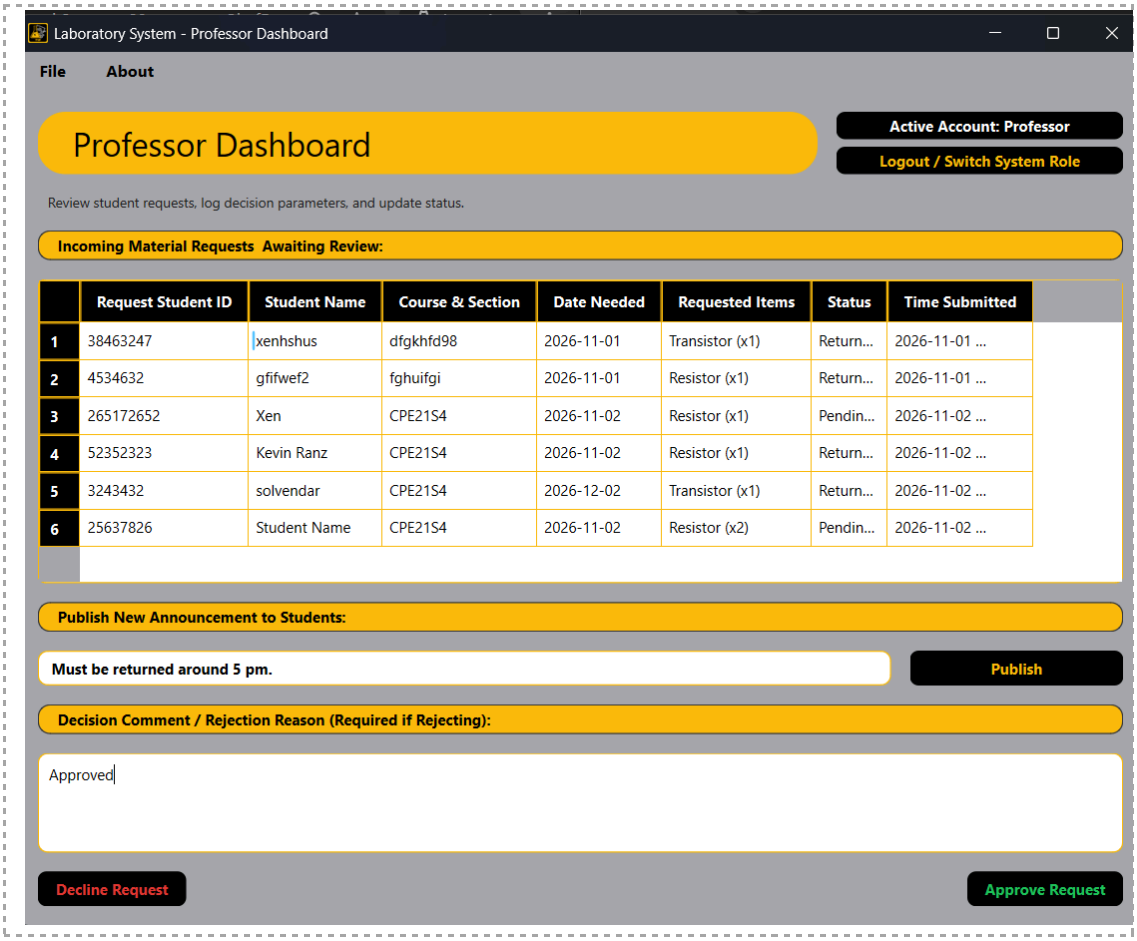


Figure 5. Professor Dashboard:

Inside the dashboard, it contains all the data inside the slip of the students that been filled up, including the date and time it submitted, it also contains an announcement that can be published and seen of the students in the students dashboard and a comment that if the request items would be approved or rejected but need to click the grid table before clicking it

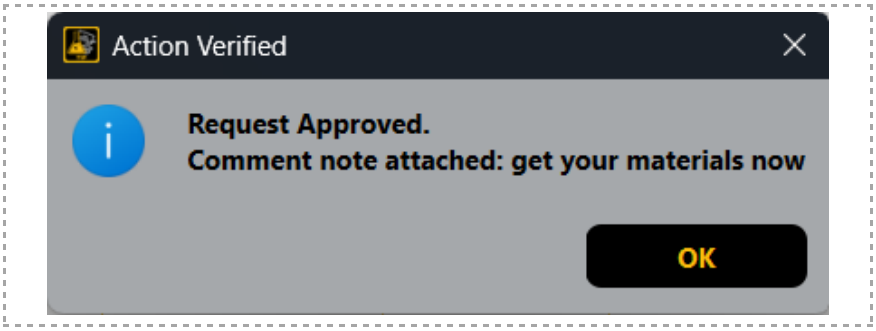


Figure 6. Professor Dashboard:

Displays a notification after the decision of the professor is done with a note attached

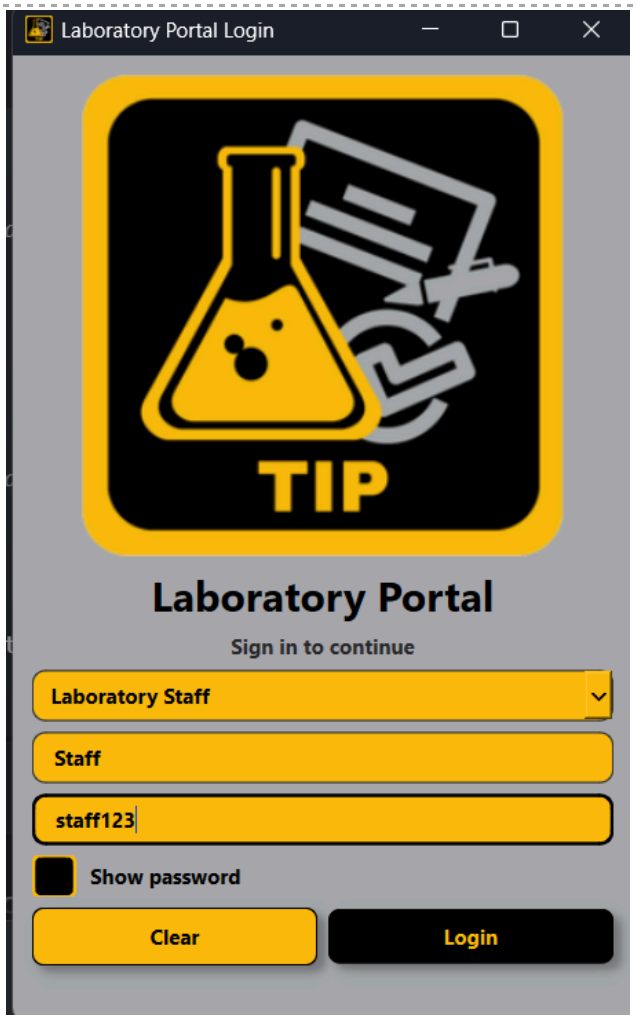


Figure 7 Staff Login:

When the user Login as a Laboratory Staff, it should be matched the role, username, and password as a Staff and it will be directed to the Staff Dashboard

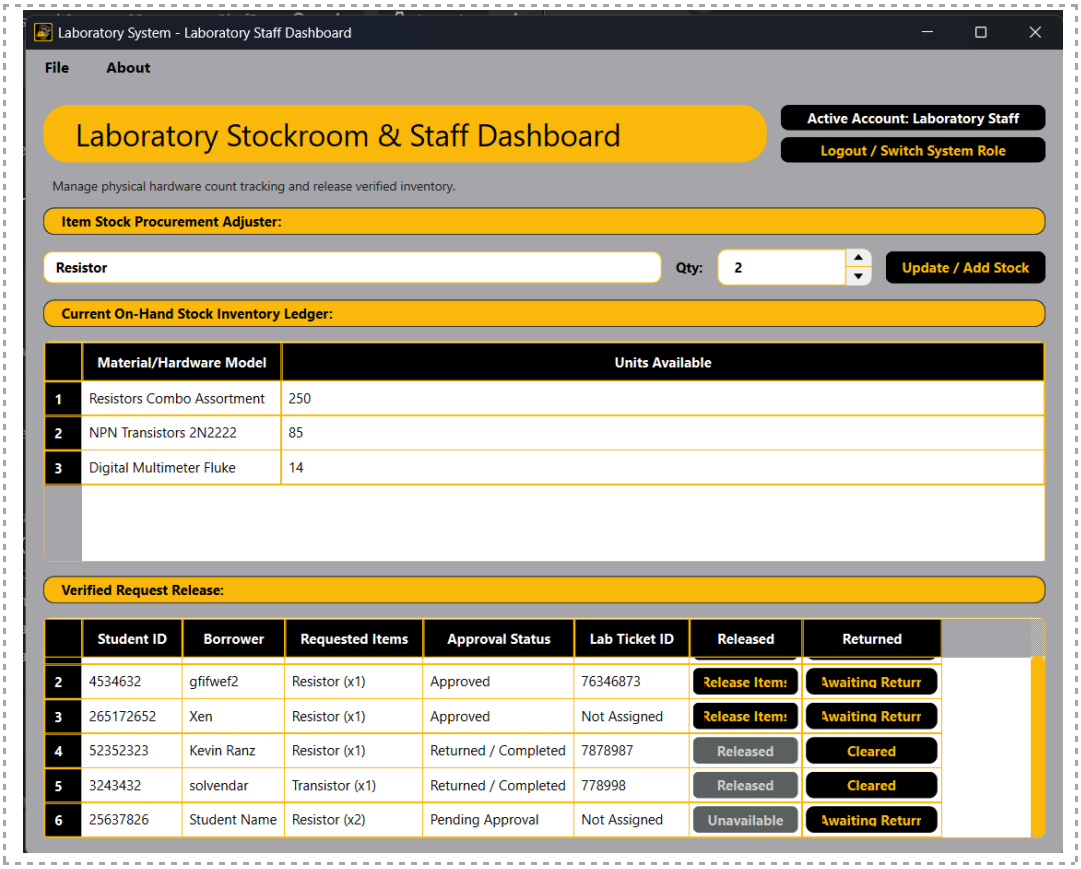


Figure 8: Staff Dashboard:

Contains all the list of the students that requested, and staff can add materials that the students requested before releasing it and providing a laboratory ID number for each student, and also a return to make the students who borrowed that material and already returned it to be cleared.

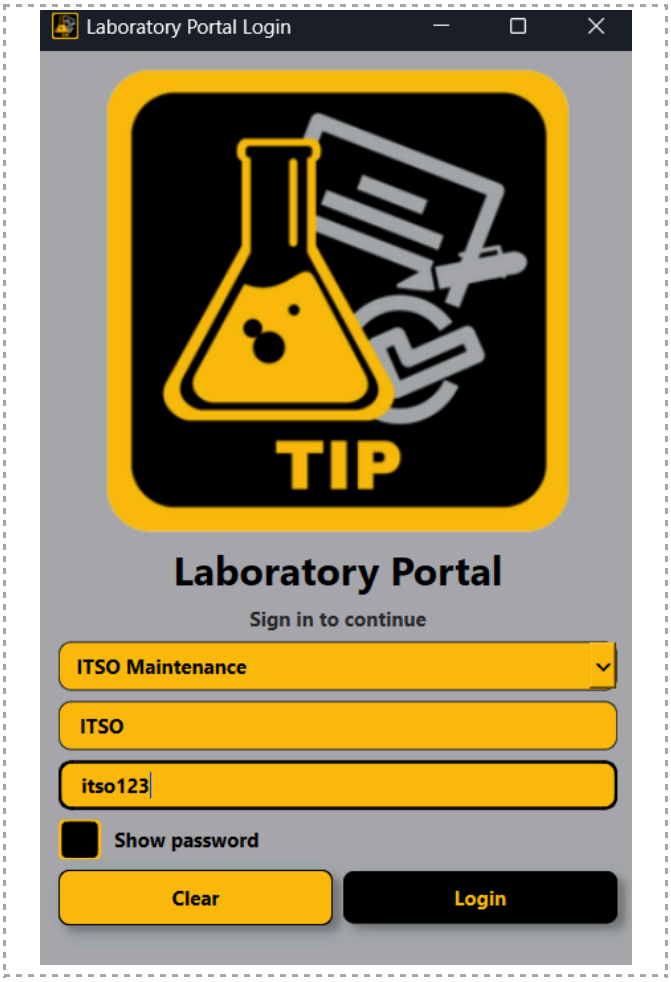
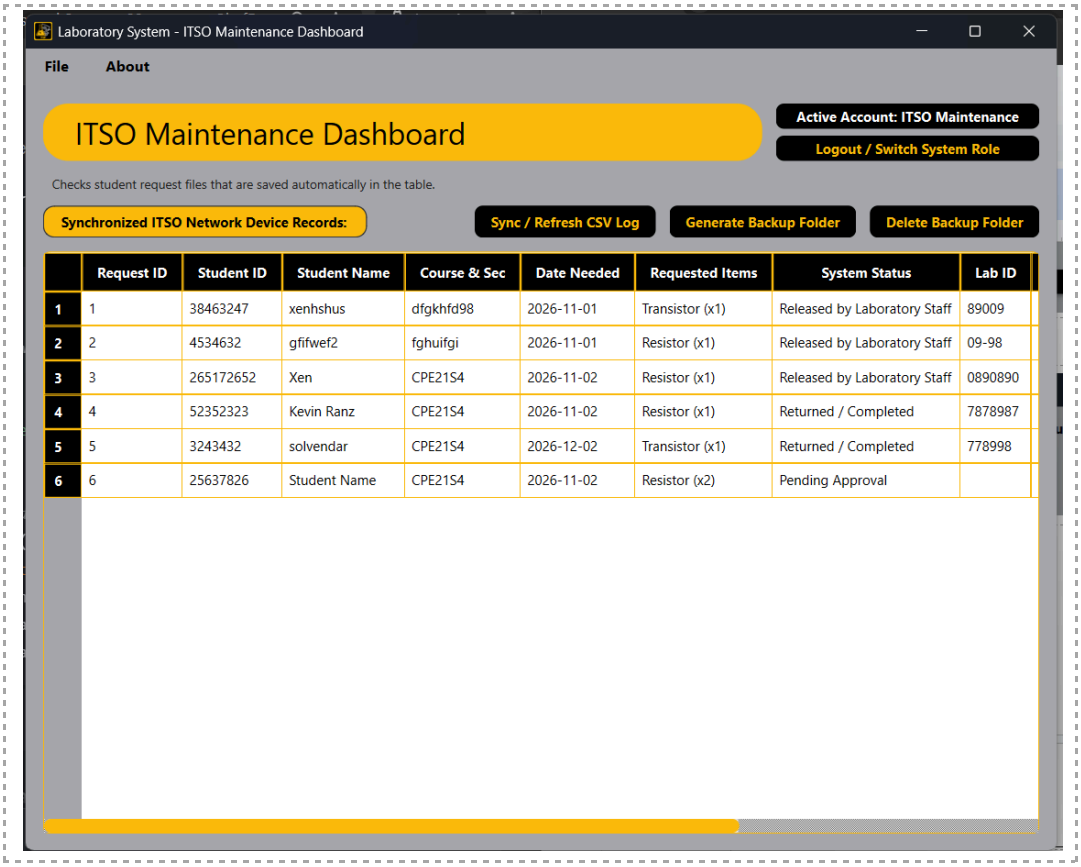


Figure 9: ITSO Login

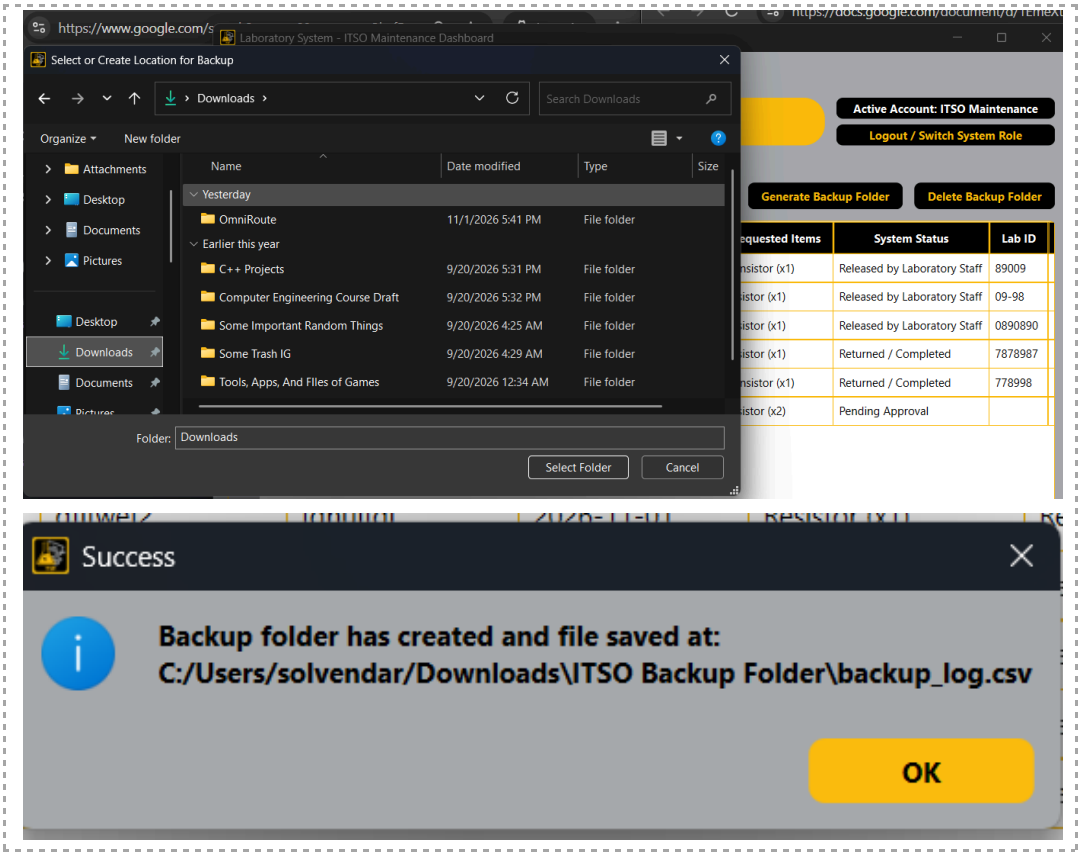
When the user Login as a ITSO Maintenance, it should be matched the role, username, and password as a ITSO Maintenance and it will be directed to the ITSO Dashboard



	Request ID	Student ID	Student Name	Course & Sec	Date Needed	Requested Items	System Status	Lab ID
1	1	38463247	xenhshus	dfgkhfd98	2026-11-01	Transistor (x1)	Released by Laboratory Staff	89009
2	2	4534632	gfifwef2	fghuifgi	2026-11-01	Resistor (x1)	Released by Laboratory Staff	09-98
3	3	265172652	Xen	CPE21S4	2026-11-02	Resistor (x1)	Released by Laboratory Staff	0890890
4	4	52352323	Kevin Ranz	CPE21S4	2026-11-02	Resistor (x1)	Returned / Completed	7878987
5	5	3243432	solvendar	CPE21S4	2026-12-02	Transistor (x1)	Returned / Completed	778998
6	6	25637826	Student Name	CPE21S4	2026-11-02	Resistor (x2)	Pending Approval	

Figure 10: ITSO Dashboard:

Contains all of the data of the students and their timestamp of submission and return of the material, the sync csv makes the csv file updated that showing on the screen of the list of the students, while the generate and delete folder creates a folder which depends on the user where destination of that folder to be put and inside was a csv/xlxs file containing the data of each students inside as a back up just in case the data is missing or got deleted. While, the delete a folder deleting only the folder that have been generated using shutil module to force the folder and the data to be deleted in the file explorer


Laboratory System - ITSO Maintenance Dashboard
Active Account: ITSO Maintenance
Logout / Switch System Role
Generate Backup Folder
Delete Backup Folder
Requested Items
System Status
Lab ID
Released by Laboratory Staff
Returned / Completed
Pending Approval
Select or Create Location for Backup
Downloads
Search Downloads
Organize
New folder
Attachments
Desktop
Documents
Pictures
Name
Date modified
Type
Size
Yesterday
OmniRoute
11/1/2026 5:41 PM
File folder
Earlier this year
C++ Projects
9/20/2026 5:31 PM
Computer Engineering Course Draft
9/20/2026 5:32 PM
Some Important Random Things
9/20/2026 4:25 AM
Some Trash IG
9/20/2026 4:29 AM
Tools, Apps, And Files of Games
9/20/2026 12:34 AM
Folder:
Select Folder
Cancel

Success
Backup folder has created and file saved at:
C:/Users/solvendar/Downloads\ITSO Backup Folder\backup_log.csv
OK

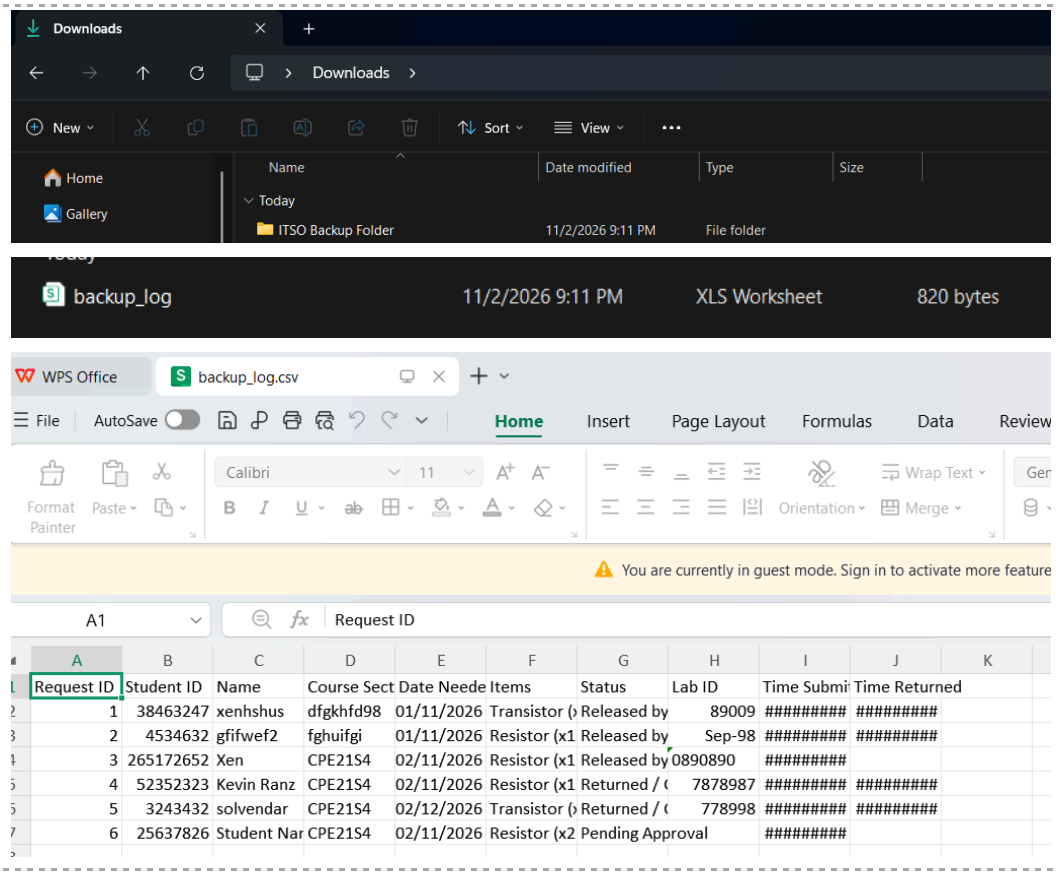


Figure 11. ITSO Dashboard:

GENERATING AND DISPLAYING THE FOLDER AND CSV DATA INSIDE

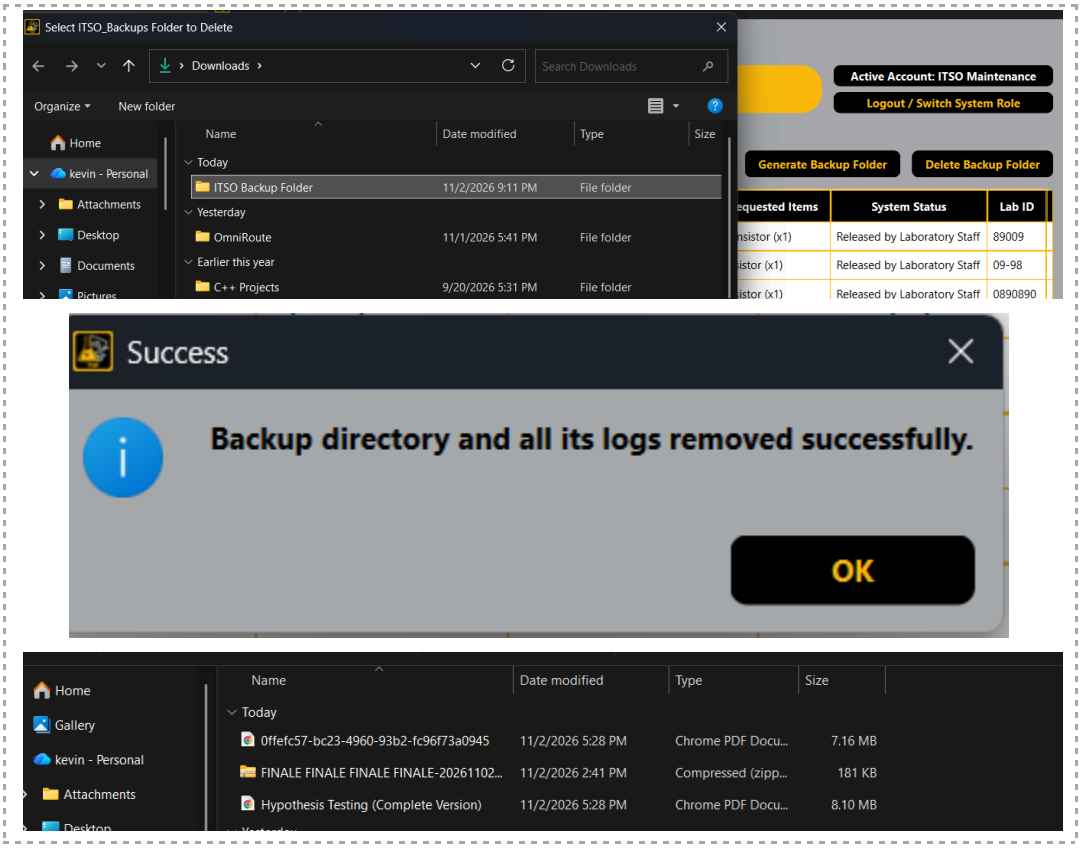


Figure 11. ITSO Dashboard:

DELETING THE FOLDER THAT HAS BEEN GENERATED

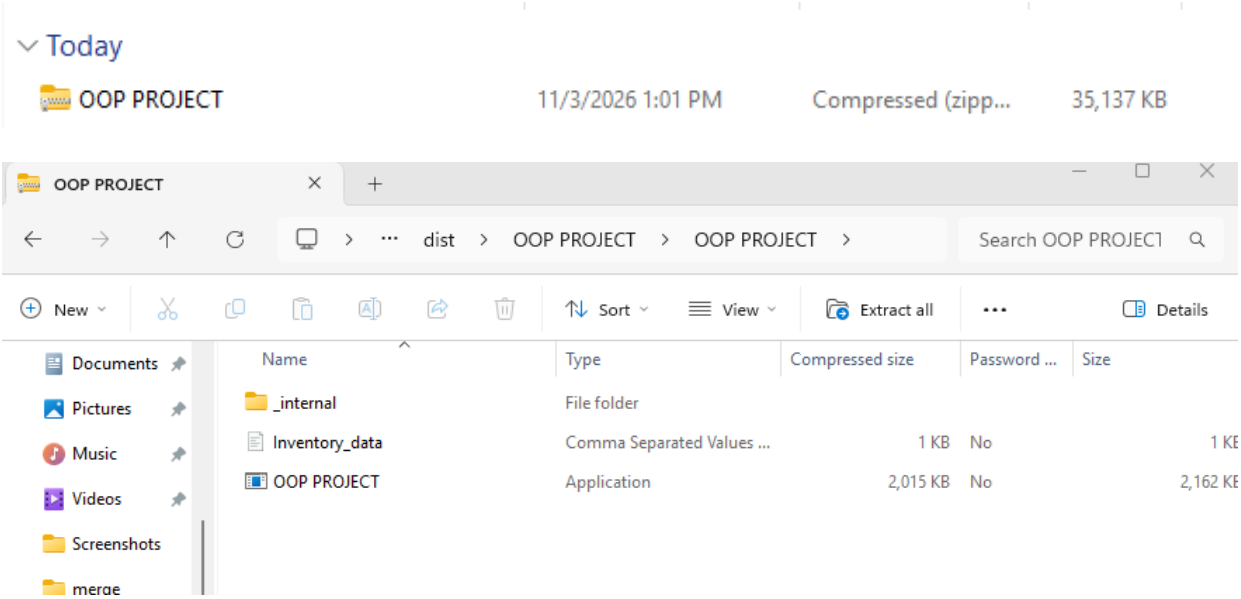


Figure 12. Make the .exe file

5.2 Test Results

#	Feature Tested	Input / Action	Expected Result	Actual Result	Pass / Fail
1	Login	Wrong password	Error message	Error message show, window shakes	Pass
2	Login	Correct username and password	Dashboard opens	Depend of the chosen role the dashboard open based on it	Pass
3	Login	Role does not match the account	Login rejected	Error message shown	Pass
4	Login	Tick Show password	Password visible	Password is show as a text	Pass
5	Login	Press Clear	fields of letter cleared	Username and password cleared	Pass
6	Student slip	Submit with blank ID or name	Warning	Validation error appeared and nothing saved	Pass
7	Student slip	Submit with no items added	Warning	Validation error appeared and nothing saved	Pass
8	Student slip	Add Item with blank material name	Warning	Warning shown and item not added	Pass
9	Student slip	Add item with a name and quantity	Items added to the list	Item appears in the table	Pass
10	Student slip	Press clear items	List emptied	All the added material was removed	Pass
11	Student slip	Submit a valid request	Request saved	Saved in the CSV log as the pending approval	Pass
12	Stock Search	Type resistor	Shows units available	It show the materials and its quantity of it	Pass
13	Professor	Approve or decline without row of table selected	Warning	Warning shown	Pass
14	Professor	Decline without a reason	Warning	Warning shown and status unchanged	Pass
15	Professor	Decline with a reason	Status rejected	Status rejected and save at the CSV log	Pass

16	Professor	Approve a request	Status Approved	The status changes to approved and it is saved	Pass
17	Professor	Publish an announcement with empty text	Warning	Warning not be sent to student	Pass
18	Professor	Publish an announcement with a text	Success	Announcement shown and transfer to student bulletin	Pass
19	Staff stock	Add quantity to an existing item	Quantity of it will increase	Units added to the material and was saved into CSV	Pass
20	Staff stock	Add with a blank name material	Warning	Warning was shown and nothing added	Pass
21	Staff stock	Add a new item component	New row will created and be added	New items appear in ledger and the CSV is updated	Pass
22	Staff Release	Release an approved request by the professor then enter the LAB ID	Status released and the inventory stock deducted	Status released and available units decrease	Pass
23	Staff Release	Release more than the available stock	Release blocked	Warning shown and nothing release	Pass
24	Staff Release	Cancel the lab ticket ID dialog	Release will be cancelled	Warning shown and nothing release	Pass
25	Staff Return	Press confirm return	Stock restored	Status of returned will be cleared and units added back	Pass
26	Student Tracking	View a completed request	Button says returned	The button shows returned and units added back	Pass
27	ITSO	Press Generate Backup Folder	Backup folder created	Backup Folder will be stored at backup_log.csv	Pass
28	ITSO	Press Delete Backup Folder	Folder was removed	Folder will be deleted	Pass
29	Menu Bar	File can do the Logout and Exit and About is who created it	Each of works	Back to its login or about dialog shows who created or program closes	Pass

30	File Saving	Close and reopen the program	Data is still there	The data of slips and stock is stored in CSV files.	Pass

5.3 Discussion

Specific Objective	Met? (Yes / Partly / No)	Evidence (Figure or Test No.)
Role-based logins, dashboards and the full borrow-slip workflow	Yes	Figures 1 to 11, Tests 1 to 5, 11, 16, 22, 25
Validate user entries and give feedback	Partly	Tests 1, 3, 6, 7, 8, 14, 17, 20, 23
Store and load slips and inventory in CSV files	Yes	Tests 11, 15, 16, 19, 21, 27, 30
Apply encapsulation, abstraction, inheritance and polymorphism	Yes	Sections 4.2 to 4.6
Package as a Windows .exe	Yes	Section 2.3 and Section 3.5

Discussion: The application met most of its objectives. As shown in Figures 1 to 10, each role has its own login and its own dashboard, and a slip moves from the student to the professor, then to the staff, and is saved for ITSO. Tests 11, 16, 22 and 25 show that the status changes at every step and that the stock goes down on release and back up on return. Because all 30 tests passed, the main workflow works as expected.

For Objective 2, the validation works for blank fields, an empty item list, a blank material name, a missing decline reason, an empty announcement, and releasing more than the available stock (Tests 6, 7, 8, 14, 17, 20 and 23). We marked it “Partly” because the Student ID and name are only checked if they are empty. The program does not check if the ID has the right format or if the name has numbers. The usernames and passwords are also fixed in the code, so there is no real registration, which we listed in the delimitations.

For Objective 3, Test 30 shows that the slips and the stock are still there after closing and reopening the program. This means the CSV files work as storage. The backup and delete features in Tests 27 and 28 give ITSO a way to protect the records. For Objective 4, the code uses a parent and child file class, overridden read() and write() methods, and separate classes for data, logic and layout. Objective 5 is met because the .exe runs without Python installed. Some things were not fully done. The professor sees all requests, not only the ones assigned to them. The app also works on one computer only, so two users on different computers cannot use it at the same time. These were listed in the delimitations.

5.4 Challenges Encountered

Challenge	Cause	Solution
The .exe could not find style.qss and the icons	The files were not bundled with the program	Added the files when packaging and used a path based on __file__

The student, professor and staff screens showed different data	Each dashboard had its own copy of the requests and stock	Made one shared list in main.py and passed it to every dashboard, and only changed it in place
The CSV header row was loaded as a student request	The file has a header line on the first row	Added a check for “Request ID” so the header is skipped when loading
The stock file could be damaged or could crash the program on save	The CSV can be open in another program, or the program can close while saving	Saved to a temporary file first, then swapped it in, and added try and except with a warning message
The spin box arrows disappeared after using the QSS style	The style sheet hid the default arrows	Made the ArrowSpinBox class that draws its own up and down triangles
The class name QMainWindow caused confusion	Our main class used the same name as the PyQt class	Renamed it to LabPortalApp and used the real QMainWindow only for the menu bar
The status in the CSV did not update correctly	The CSV handler only adds lines at the end of the file	Deleted the old file and wrote all the rows again when a status changed

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

The group built a PyQt6 desktop application called the Laboratory Equipment Borrowing and Inventory Management System. It replaces the paper borrowing slip with a digital process. A student submits a request, the professor approves or declines it, the laboratory staff release and receive the items, and ITSO keeps a record of every slip in a CSV file. The program also keeps the inventory updated when items are added, released and returned. It has a login for each role, a menu bar, dialogs, a QSS theme, and it was packaged as a Windows .exe. All 30 test cases passed, and four of the five objectives were fully met. Objective 2 was partly met because the student ID and name are only checked if they are empty.

The application answers the problem in the paper. Students no longer need to fill out a slip by hand or line up just to find out an item is not available, because they can search the stock and track the request. Professors can approve from the dashboard, and the staff can see the approved requests. ITSO has an organized record with the submitted and returned times, so it is easier to know who borrowed what. The OOP principles were also applied through the file reader classes, the data classes, the inventory manager, and the separate UI and logic classes.

6.2 Recommendations

- Add email or SMS notifications. Students would know right away when a request is approved, declined or ready for release, so they don’t need to keep checking the app.
- Add a toolbar button for common actions, and a log for damaged or missing items. This would help the staff record problems when the items are returned.
- Let each professor see only their assigned requests. Right now a professor sees every request, so this would remove confusion and protect the privacy of students.

- Add stronger checks for the student ID, name and section. Checking the format would stop wrong or fake data from going into the ITSO records.
- Add a database such as SQLite. CSV files are easy to open and change by mistake, and they can be damaged when two programs use them. A database is safer and faster when there are many records.
- Add real accounts and registration. The usernames and passwords are fixed in the code. Real accounts would show which student or professor is using the app, and passwords would be safer.
- Make the program work on many computers. The app runs on one computer only. A network or server version would let students request from their own devices and the staff release at the lab at the same time.

7. REFERENCES

PyInstaller Development Team. (n.d.). PyInstaller manual. https://pyinstaller.org/

Python Software Foundation. (n.d.). csv: CSV file reading and writing. Python 3 documentation. https://docs.python.org/3/library/csv.html

Python Software Foundation. (n.d.).shutil: High-level file operations. Python 3 documentation. https://docs.python.org/3/library/shutil.html

Riverbank Computing. (n.d.). PyQt6 reference guide. https://www.riverbankcomputing.com/static/Docs/PyQt6/

The Qt Company. (n.d.). Qt style sheets reference. https://doc.qt.io/qt-6/stylesheet-reference.html

APPENDICES

Appendix A. Source Code

A.1 main.py

```
import sys
import os
from datetime  import datetime
from pathlib import Path
from PyQt6.QtGui import QIcon
from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
                             QLabel, QMainWindow, QMessageBox)
from PyQt6.QtCore import Qt
from Data_Model   import BorrowRequestData
from CSVFileReaderWriter  import CSVFileReaderWriter
from InventoryManager import InventoryManager
from LoginSystem  import LoginWindow
from Student   import StudentDashboard
from Prof import ProfessorDashboard
from Staff import StaffDashboard
from ITSO import ITSODashboard
```

```
# renamed from QMainWindow so it no longer replaces PyQt's QMainWindow class
class LabPortalApp:  # renamed from QMainWindow
    def __init__(self):
        # saved data that is shared between all screens
        self.shared_requests = []
        self.shared_announcements = []
        # inventory is loaded from inventory_data.csv and saved back whenever staff add, release or
return items
        self.inventory =  InventoryManager()
        self.shared_inventory  = self.inventory.items

        self.csv_filepath  = "ITSO_Maintenance_data.csv"

        if os.path.exists (self.csv_filepath):
            try:
                csv_handler =  CSVFileReaderWriter()
                text_line =   csv_handler.read (self.csv_filepath)

                if text_line and str(text_line[0][0]).strip()   == "Request ID":
                    rows =  text_line[1:]
                else:
                    rows =   text_line

                for row in rows:
                    if not row or len(row)  < 10:
                        continue

                    try:
                        date_needed   = datetime.strptime(row[4], "%Y-%m-%d").date()
                    except:
                        date_needed  = datetime.now().date()

                    req =   BorrowRequestData(
                        request_id=int (row[0]),
```

```
                        student_id=str (row[1]),
                        name=str  (row[2]),
                        course_section=str (row[3]),
                        date_needed=date_needed,
                        items=str (row[5])
                    )

                    req.status   = str  (row[6])
                    req.lab_id = str (row[7])
                    req.timestamp_submitted  =  str(row[8])
                    req.timestamp_returned  = str(row[9])

                    self.shared_requests.append(req)
            except Exception as e:
                print(f"Could not load old data on startup: {e}")

        self.login_win   = LoginWindow()
        self.active_dashboard  = None
        self.master_container   = None

        # disconnect and connect the login button action
        self.login_win.login_btn.clicked.disconnect()
        self.login_win.login_btn.clicked.connect(self.process_system_login)
        self.login_win.show()

    def process_system_login (self):
        # checks the login details and opens the correct dashboard
        user   = self.login_win.username.text().strip()
        password  = self.login_win.password.text()
        role  = self.login_win.choose_role.currentText()

        is_valid  = False
        if role   == "Student" and user  == "Student" and password == "student1234":
            is_valid = True
        elif role  == "Professor" and user   == "Professor" and password == "prof123":
            is_valid = True
```

```
        elif role  == "Laboratory Staff" and user == "Staff" and password   == "staff123":
            is_valid = True
        elif role == "ITSO Maintenance" and user   == "ITSO" and password  == "itso123":
            is_valid  = True


        if is_valid:
            self.login_win.set_style_property (self.login_win.username, "error", False)
            self.login_win.set_style_property  (self.login_win.password, "error", False)
            self.login_win.set_style_property (self.login_win.status, "state", "success")


            self.login_win.hide()
            self.launch_dashboard_workspace(role)
        else:
            self.login_win.set_style_property (self.login_win.username, "error", True)
            self.login_win.set_style_property  (self.login_win.password, "error", True)
            self.login_win.set_style_property (self.login_win.status, "state", "error")
            self.login_win.status.setText ("Invalid username or password for this role.")
            self.login_win.shake()


    def launch_dashboard_workspace ( self, role):
        # start the correct user role and pass the shared memory data
        if role   == "Student":
            self.active_dashboard  = StudentDashboard(
                shared_requests_list =self.shared_requests,
                shared_announcements  =self.shared_announcements,
                shared_inventory =self.shared_inventory
            )
        elif role  == "Professor":
            self.active_dashboard   = ProfessorDashboard(
                shared_requests_list =self.shared_requests,
                shared_announcement  =self.shared_announcements
            )
        elif role   == "Laboratory Staff":
            self.active_dashboard = StaffDashboard  (shared_requests_list=self.shared_requests,
inventory_manager=self.inventory)
        elif role  == "ITSO Maintenance":
```

```
            self.active_dashboard  = ITSODashboard()


        # black pill that shows which account is logged in (white text)
        title_tag   = QLabel (f"Active Account: {role}")
        title_tag.setAlignment (Qt.AlignmentFlag.AlignCenter)
        title_tag.setFixedWidth( 230)  # (change this number to resize the pill)
        title_tag.setStyleSheet("""
            background-color: #000000;
            color: #ffffff;
            border-radius:  7px;
            padding: 3px 10px;
            font-size: 12px;
            font-weight: bold;
        """)

        # black pill button with yellow text to log out or switch role
        switch_btn  = QPushButton  ("Logout / Switch System Role")
        switch_btn.setCursor (Qt.CursorShape.PointingHandCursor)
        switch_btn.setFixedWidth(  230)  # (change this number to resize the pill)
        switch_btn.setStyleSheet("""
            QPushButton {
                background-color: #000000;
                color: #fabb0d;
                border-radius: 7px;
                padding: 3px 10px;
                font-size: 12px;
                font-weight: bold;
            }
            QPushButton:hover {
                background   -  color: #fabb0d;
                color: #000000;
            }
        """)
        switch_btn.clicked.connect  (self.return_to_login_screen)

        # stacks the account tag on top of the logout button
```

```
        account_box = QVBoxLayout()
        account_box.setSpacing( 6)
        account_box.addWidget (title_tag)
        account_box.addWidget  (switch_btn)

        # finds the yellow title pill of the dashboard and puts the account box beside it
        dash_layout   = self.active_dashboard.layout()
        page_title  = self.active_dashboard.findChild(QLabel, "pageTitle")
        title_index  = dash_layout.indexOf(page_title)
        dash_layout.removeWidget (page_title)

        # header row = yellow title on the left, account box on the right
        header_row  = QHBoxLayout()
        header_row.addWidget (page_title, 1)
        header_row.addLayout  (account_box)
        dash_layout.insertLayout (title_index, header_row)

        # start: real Qt main window that holds the menu bar; the dashboard is its central widget
        self.master_container  = QMainWindow()
        self.master_container.setCentralWidget  (self.active_dashboard)

        # menu bar: File and About menus
        menu_bar =   self.master_container.menuBar()

        file_menu   = menu_bar.addMenu ("File")
        file_menu.addAction  ("Logout / Switch System Role", self.return_to_login_screen)
        file_menu.addSeparator()
        file_menu.addAction  ("Exit", QApplication.quit)

        about_menu = menu_bar.addMenu  ("About")
        about_menu.addAction ("Group 3 - CPE21S4", self.show_about)

        self.master_container.setWindowTitle (f"Laboratory System - {role} Dashboard")
        self.master_container.resize(  900 , 700)
        self.master_container.show()
```

```
    # whole method: dialog opened from About > Group 3
    def show_about (self):
        # dialog showing the group name and members
        QMessageBox.information(
            self.master_container, "About",
            "Laboratory Equipment Borrowing and Inventory Management System\n\n"
            "Group 3 - CPE21S4\n"
            "Acelajado, John Vince D.\n"
            "Bal, Kevin Ranz J.\n\n"
            "Instructor: Engr. Jimlord M. Quejado"
        )

    def return_to_login_screen (self):
        # order changed: show the login screen first, then close the dashboard
        self.login_win.username.clear()
        self.login_win.password.clear()
        self.login_win.status.setText("")
        self.login_win.show()
        if self.master_container:
            self.master_container.close()



if __name__  == "__main__":
    if sys.platform   == "win32":
        import ctypes
        ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID  ("tip.laboratory.portal")

    app = QApplication  (sys.argv)
    app.setWindowIcon(QIcon(str(Path(__file__).with_name ("app_icon.ico"))))
    qss_filepath = Path(__file__).with_name("style.qss")
    if qss_filepath.exists():
        app.setStyleSheet (qss_filepath.read_text())

    window = LabPortalApp()
    sys.exit(app.exec())
```

A.2 LoginSystem.py

```
import sys
from PyQt6.QtCore import Qt, QPropertyAnimation, QEasingCurve, QPoint
from PyQt6.QtGui import QColor, QPixmap
from PyQt6.QtWidgets import (QApplication, QWidget, QLabel, QLineEdit, QComboBox,
QPushButton, QCheckBox, QVBoxLayout, QHBoxLayout, QGraphicsDropShadowEffect)
from pathlib import Path

class LoginWindow(QWidget):
    def __init__(self):
        super().__init__()

        # starts the  smooth fade-in animation  when  the  window  opens
        self.setWindowOpacity(0.0)
        self.fade  =  QPropertyAnimation(self, b"windowOpacity")
        self.fade.setDuration(800)
        self.fade.setStartValue(0.0)
        self.fade.setEndValue(1.0)
        self.fade.setEasingCurve(QEasingCurve.Type.InOutQuad)
        self.fade.start()

        # window  main properties
        self.setWindowTitle("Laboratory Portal Login")
        self.resize(360, 380)

        # text  headings and title widgets
        self.title  = QLabel("Laboratory Portal")
        self.subtitle = QLabel("Sign in to continue")

        # dropdown selection to let  users pick  their role in  the system
        self.choose_role  = QComboBox()
        self.choose_role.addItems(["Student", "Professor", "Laboratory Staff", "ITSO Maintenance"])

        # text  input  boxes  for username  and  password
        self.username = QLineEdit()
```

```
        self.username.setPlaceholderText("Username")
        self.password = QLineEdit()
        self.password.setPlaceholderText("Password")
        self.password.setEchoMode(QLineEdit.EchoMode.Password)

        # control checkboxes and action buttons
        self.show_password  = QCheckBox("Show password")
        self.login_btn  = QPushButton("Login")
        self.clear_btn =  QPushButton("Clear")
        self.status  = QLabel("")

        # shadow  effects  for clear  button
        shadow  = QGraphicsDropShadowEffect()
        shadow.setOffset(4, 4)
        #accessing the  x(right) and  y(down) coordinates layout  of  how far  the is the  shadow
away  from  the each button
        shadow.setColor(QColor(0, 0, 0, 80))
        # controlling of  how blur  or sharp the  edges of  the shadows  are
        shadow.setBlurRadius(15)
        # calling  the  setgraphicseffect to display  the customized  widget
        self.clear_btn.setGraphicsEffect(shadow)

        # shadow  effects  for  login  button
        shadow2  = QGraphicsDropShadowEffect()
        shadow2.setColor(QColor(0, 0, 0, 80))
        #accessing the x(right) and  y(down) coordinates layout of how far  the is the shadow away
from the  each  button
        shadow2.setOffset(4, 4)
        # controlling  of  how blur  or sharp  the  edges  of the  shadows are
        shadow2.setBlurRadius(15)
        # calling  the setgraphicseffect to display  the customized widget
        self.login_btn.setGraphicsEffect(shadow2)

        # changes the mouse cursor to a  pointer hand when hovering over  buttons
        self.login_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        self.clear_btn.setCursor(Qt.CursorShape.PointingHandCursor)
```

```
        # assigns styling property  tags  for  the stylesheet
        self.title.setObjectName("title")
        self.subtitle.setObjectName("subtitle")
        self.login_btn.setObjectName("loginBtn")
        self.clear_btn.setObjectName("clearBtn")
        self.status.setObjectName("status")
        self.title.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.subtitle.setAlignment(Qt.AlignmentFlag.AlignCenter)

        # arranges action buttons  side  by side in a  row layout line
        buttons = QHBoxLayout()
        buttons.addWidget(self.clear_btn)
        buttons.addWidget(self.login_btn)

        # tip logo shown above  the "laboratory portal" title (app_icon.png sits  next to  this
file)
        self.logo = QLabel()
        self.logo.setObjectName("logo")
        logo_pixmap = QPixmap(str(Path(__file__).with_name("app_icon.png")))
        if not logo_pixmap.isNull():
            self.logo.setPixmap(logo_pixmap.scaledToWidth(280,
Qt.TransformationMode.SmoothTransformation))
        self.logo.setAlignment(Qt.AlignmentFlag.AlignCenter)

        # stacks all  elements vertically straight  down the  page layout
        layout  = QVBoxLayout()
        layout.addWidget(self.title)
        layout.addWidget(self.subtitle)

        layout = QVBoxLayout()
        layout.addWidget(self.logo)
        layout.addWidget(self.title)
        layout.addWidget(self.subtitle)
        layout.addWidget(self.choose_role)
        layout.addWidget(self.username)
        layout.addWidget(self.password)
        layout.addWidget(self.show_password)
```

```
        layout.addLayout(buttons)
        layout.addWidget(self.status)
        self.setLayout(layout)

        # signal buttons  to  toggle for  login, clear, and show password
        self.login_btn.clicked.connect(self.login)
        self.clear_btn.clicked.connect(self.clear)
        self.show_password.toggled.connect(self.toggle_password)

    # shows  and hide  the password  that depends  on  the user's  choice
    def toggle_password(self, checked):
        if checked:
            self.password.setEchoMode(QLineEdit.EchoMode.Normal)
        else:
            self.password.setEchoMode(QLineEdit.EchoMode.Password)

    # checks  if  the  username  and password match on  the right role
    def login(self):
        user   = self.username.text().strip()
        password  =  self.password.text()
        role = self.choose_role.currentText()

        # checks  if  the typed username  and  password match  the  selected  role  options
        is_valid = False
        if role == "Student" and user == "Student" and password  ==   "student1234":
            is_valid = True
        elif role == "Professor" and user == "Professor" and password == "prof123":
            is_valid = True
        elif role == "Laboratory Staff" and user == "Staff" and password  ==   "staff123":
            is_valid = True
        elif role  == "ITSO Maintenance" and user == "ITSO" and password   ==  "itso123":
            is_valid = True

        # checks  if  the  username and password match on the selected  role
        if is_valid:
            self.set_style_property(self.username, "error", False)
```

```
            self.set_style_property(self.password, "error", False)
            self.set_style_property(self.status, "state", "success")
            self.status.setText(f"Welcome, {user} ({role})!")
        else:
            self.set_style_property(self.username, "error", True)
            self.set_style_property(self.password, "error", True)
            self.set_style_property(self.status, "state", "error")
            self.status.setText("Invalid username or password for this role.")
            self.shake()

    # resets  login input  text elements  back to default states
    def clear(self):
        self.username.clear()
        self.password.clear()
        self.status.setText("")
        self.username.setFocus()
        self.set_style_property(self.username, "error", False)
        self.set_style_property(self.password, "error", False)
        self.set_style_property(self.status, "state", "")

    def set_style_property(self, widget, name, value):
        widget.setProperty(name, value)
        widget.style().unpolish(widget)
        widget.style().polish(widget)

    # shakes  the window frame left and right to signal  that  the  password  is  incorrect
    def shake(self):
        if hasattr(self, "shake_anim") and self.shake_anim.state()  ==
QPropertyAnimation.State.Running:
            return

        start = self.pos()
        offsets = [0,  - 10, 10, -8, 8, -4, 4, 0] # pixels left (-) and right (+)
        self.shake_anim = QPropertyAnimation(self, b"pos")
        self.shake_anim.setDuration(400)
        for i, dx in enumerate(offsets):
```

```
            step = i / (len(offsets) - 1)
            self.shake_anim.setKeyValueAt(step, start + QPoint(dx, 0))
        self.shake_anim.start()
```

A.3 Student.py

```
from PyQt6.QtWidgets import QWidget, QMessageBox, QTableWidgetItem, QPushButton
from Student_UI import StudentDashboardApp
from pathlib import Path
from Data_Model import BorrowRequestData
from CSVFileReaderWriter import CSVFileReaderWriter
import sys
import os
from datetime import datetime
class StudentDashboard(QWidget):
    def __init__( self, shared_requests_list=None , shared_announcements= None , shared_inventory =
None):
        super().__init__()

        # saves the  student requests list if  it exists, or makes  a  new  empty  list  if it is
missing
        self.shared_requests  = shared_requests_list if shared_requests_list is not None else []

        # saves  the  professor  announcements  list if it  exists, or makes  a  new empty  list
if  it is missing
        self.shared_announcements   = shared_announcements if shared_announcements is not None else
[]

        # saves  the  items list  from  the  stockroom if it  exists, or makes a  new  empty list
if  it is  missing
        self.shared_inventory = shared_inventory if shared_inventory is not None else[]

        # creates  an instance of  the  interface layout
        self.ui  =   StudentDashboardApp()
        self.ui.setupUi (self)

        self.display_request()  # display the student's own requests in the tracking table
        #signals
```

```
        self.ui.add_item_btn.clicked.connect  (self.add_item_to_table)
        self.ui.clear_items_btn.clicked.connect(self.clear_table_items)
        self.ui.submit_request_btn.clicked.connect (self.process_submission)
        self.ui.stock_search_input.textChanged.connect(self.search_stock) # connects the typing
input box to the search function


        # runs  the professor  list update function  whenever the  program dropdown  changes
        self.ui.program.currentTextChanged.connect (self.update_professor_list)


        # automatically  updates the  dashboard display items list on  startup
        self.load_announcements_data()


        # sets  up  the  correct professors for  the  default  selected program  on  startup
        self.update_professor_list()


    def display_request (self):
        # clears out  the  table  before displaying  new data
        self.ui.tracking_table.setRowCount  (0)
        student_id  = self.ui.student_id.text().strip()


        # loops  through  the  shared requests list  to  fill the display  rows
        for idx, req in enumerate (self.shared_requests):
        # if  student_id  field  is  empty, show all for  testing, otherwise filter by active
student
            if  student_id and req.student_id != student_id:
                continue


            row  = self.ui.tracking_table.rowCount()
            self.ui.tracking_table.insertRow(row)
            self.ui.tracking_table.setItem(row, 0, QTableWidgetItem(str(req.request_id)))
            self.ui.tracking_table.setItem  (row, 1, QTableWidgetItem(req.items))
            self.ui.tracking_table.setItem(row, 2, QTableWidgetItem(req.status))
            self.ui.tracking_table.setItem (row, 3, QTableWidgetItem(req.lab_id if req.lab_id else
"N   / A"))


            return_btn   =   QPushButton ("Return to Staff")
            return_btn.setObjectName("addItemBtn") # uses the same style as the add item button
```

```
            if req.status    ==  "Released by Laboratory Staff":
                return_btn.setEnabled(True)
            else:
                return_btn.setEnabled(False)
                if req.status    ==  "Pending Return Confirmation":
                    return_btn.setText("Awaiting Return Confirmation")
                elif req.status    == "Returned/Completed":
                    return_btn.setText  ("Returned")
                else:
                    return_btn.setText ("Not Ready for Return")

            return_btn.setProperty ("row_index", idx)
            return_btn.clicked.connect  (self.handle_release_return_click)
            self.ui.tracking_table.setCellWidget (row , 4 , return_btn)


    def handle_release_return_click (self):
        # identifies  which  button was clicked and  reads its  row  number
        button =  self.sender()
        if button:
            target_row = button.property  ("row_index")
            request = self.shared_requests[target_row]


            request.status = "Pending Return Confirmation" # updates the  status to alert the staff
that the student is returning the items
            request.timestamp_returned   = datetime.now().strftime("%Y-%m-%d %H:%M:%S")


            self.sync_local_to_csv_data()  # save the updated request status to the CSV file
            QMessageBox.information( self, "Return Initiated", f"Materials for Request
#{request.request_id} are marked for return. Please surrender them to the laboratory room along
with Lab Ticket ID: {request.lab_id}.")
            self.display_request()



    def sync_local_to_csv_data (self):
        csv_filepath = "ITSO_Maintenance_data.csv"
        csv_handler = CSVFileReaderWriter()
```

```
        csv_handler.write( csv_filepath, ["Request ID", "Student ID", "Name", "Course Section",
"Date Needed", "Items", "Status", "Lab ID", "Time Submitted", "Time Returned"])
        for req in self.shared_requests:
            record_row   =   [
                str(req.request_id), req.student_id, req.name, req.course_section,
                str(req.date_needed), req.items, req.status, req.lab_id,
                req.timestamp_submitted, req.timestamp_returned
            ]
            csv_handler.write( csv_filepath, record_row)


    def load_announcements_data (self):
        # clears out the  list view widget completely  before showing  fresh notes
        self.ui.announcements_list.clear()


        # loops  through  the shared announcements list to fill the  display  rows
        for info in self.shared_announcements:
            self.ui.announcements_list.addItem(f"[{info.time_posted}] {info.author}:
{info.message}")


    # shows the  materials that match  the typed text  together with  how many are available
    def search_stock (self):
        self.ui.stock_results.clear()
        text = self.ui.stock_search_input.text().strip().lower()
        if not text:
            return
        if not self.shared_inventory:
            self.ui.stock_results.addItem  ("No inventory data received yet.")
            return
        for stock in self.shared_inventory:
            if text in stock.material_name.lower():
                self.ui.stock_results.addItem(f"{stock.material_name}   -
{stock.quantity_available} available")
        if self.ui.stock_results.count() == 0:
            self.ui.stock_results.addItem ("No matching material found.")


    def update_professor_list  (self):
```

```
        self.ui.professor.clear()

        program = self.ui.program.currentText()

        if "Computer Engineering" in program:

            self.ui.professor.addItems (["Engr. Jimlord Quejado", "Engr. Menchie Rosales", "Engr.
Ji Han Gang", "Engr. Lloyd Pornobi"])

        elif "Electronics Engineering" in program:

            self.ui.professor.addItems  (["Engr. Ryan Alimuin", "Engr. Oliver John Alajas"])

        elif "Electrical Engineering" in program:

            self.ui.professor.addItems  (["Dr. Christine Ann Teodoro Alquero", "Engr. Vhander
Ramos"])


    def add_item_to_table(self):

        name   = self.ui.material_name_input.text().strip()

        qty   =   str(self.ui.quantity_spin.value())


        # stops  the  action and shows a warning  popup  if the  material name is  blank

        if not name:

            QMessageBox.warning  (self, "Validation Error", "Please type a material name first.")

            return


        # gets  the total rows, adds  a  new  empty  row, and adds  the material name  and quantity
to  the list

        row_idx = self.ui.items_table.rowCount()

        self.ui.items_table.insertRow  (row_idx)

        self.ui.items_table.setItem (row_idx , 0, QTableWidgetItem(name))

        self.ui.items_table.setItem(row_idx , 1 , QTableWidgetItem(qty))


        # reset  the  material bar  inputs  and sets the item  count  spinbox back  to  one

        self.ui.material_name_input.clear()

        self.ui.quantity_spin.setValue (1)


    def clear_table_items (self):

        self.ui.items_table.setRowCount (0)  # removes every added material item row from the
display list table


    def process_submission  (self):

        student_name =  self.ui.student_name.text().strip()

        student_id = self.ui.student_id.text().strip()
```

```
        course_section   =   self.ui.course_section.text().strip()

        date_needed  = self.ui.date_needed.date().toPyDate()

        self.display_request()

        # shows a  warning message box  if  the  name field or  any  personal  information  text
lines are blank

        if not student_name or not student_id:

            QMessageBox.warning  (self, "Validation Error", "Please provide your student ID and
Name before submitting.")

            return


        # shows  an  error message if the student  tries  to submit  a  completely empty  table

        if self.ui.items_table.rowCount()    ==  0:

            QMessageBox.warning (self, "Validation Error", "Your request list is empty. Add at
least one item first.")

            return


        # collects  all  table  item grid lines  and  bundles them  into  a  single  string

        collected_items   = []

        for r in range  (self.ui.items_table.rowCount()):

            item_name  =   self.ui.items_table.item(r, 0).text()

            item_qty   = self.ui.items_table.item(r,  1).text()

            collected_items.append (f"{item_name} (x{item_qty})")

        items_summary = ", ".join  (collected_items)


        # creates  a  new  request  object tracking code  and adds  it  to the  shared list data
array

        request_id   = len(self.shared_requests) + 1

        new_request   = BorrowRequestData(request_id,  student_id, student_name, course_section,
date_needed, items_summary)

        self.shared_requests.append(new_request)


        # gets  the active calendar day  and clock hour  to stamp the  form

        new_request.timestamp_submitted   =  datetime.now().strftime("%Y-%m-%d %H:%M:%S")


        # csv  conversion  of  slip  of  student to itso  dashboard

        csv_filepath   = "ITSO_Maintenance_data.csv"

        csv_handler   =  CSVFileReaderWriter()
```

```
        # verifies if the target backup  file  does not exist on the computer  yet
        if not os.path.exists (csv_filepath):
            # creates the spreadsheet text  file  tracking column titles as the  very  first  line
            csv_handler.write ( csv_filepath, ["Request ID" , "Student ID" , "Name", "Course Section", "Date Needed", "Items", "Status", "Lab ID", "Time Submitted", "Time Returned"])

        # collect  and  converts all the items into  a row list
        record_row = [
            str(new_request.request_id) , new_request.student_id, new_request.name,
            new_request.course_section , str(new_request.date_needed), new_request.items,
            new_request.status, new_request.lab_id, new_request.timestamp_submitted , new_request.timestamp_returned
        ]
        # saves  the rows of data directly into  the spreadsheet file
        csv_handler.write  (csv_filepath, record_row)

        # displays a  confirmation that  the request form  was submitted
        QMessageBox.information (self, "Success", f"Borrowing request submitted successfully for {student_name}!")
        self.clear_entire_form()

    def clear_entire_form (self):
        #clears  out all  text entry fields  and empties the  main table  list
        self.ui.student_id.clear()
        self.ui.student_name.clear()
        self.ui.course_section.clear()
        self.clear_table_items()
        self.display_request()
```

A.4 Student_UI.py

```
from PyQt6.QtWidgets import (
    QVBoxLayout, QHBoxLayout, QLabel, QLineEdit,
    QComboBox, QDateEdit, QSpinBox, QPushButton,
    QTableWidget, QFormLayout, QListWidget,QWidget
)
from PyQt6.QtCore import QDate, Qt
from arrowspinbox import ArrowSpinBox #NEW
```

```
class StudentDashboardApp (QWidget):
    def setupUi( self, widget):

        #include the main widget container name  so  it matches the style  sheet
        widget.setObjectName ("StudentDashboard")

        # vertical layout to  stack  elements straight down from top to  bottom
        self.main_layout  =   QVBoxLayout(widget)
        self.main_layout.setContentsMargins ( 15 , 15, 15 , 15)
        self.main_layout.setSpacing (12)

        # puts  the  main header text  label for the top of  the dashboard
        self.title = QLabel  ("Student Dashboard Workspace")
        self.title.setObjectName ("pageTitle")

        # display a smaller subtitle text explanation block below the main  title
        self.subtitle  = QLabel ("Submit a material request and follow its approval status.")
        self.subtitle.setObjectName ("pageSubtitle")
        # puts  both header text lines into the main vertical layout stack
        self.main_layout.addWidget (self.title)
        self.main_layout.addWidget (self.subtitle)

        # form  layout  that automatically  lines  up  input fields  with  labels
        self.form_layout  = QFormLayout()
        self.form_layout.setSpacing( 10)

        # input line  box  for typing a student's  id
        self.student_id  = QLineEdit()
        self.student_id.setPlaceholderText ("Input your student number here e.g. 289298213")

        # input line  box for typing the full name of the student
        self.student_name  = QLineEdit()
        self.student_name.setPlaceholderText ("Input your name here e.g. Alex Dela Cruz")

        # drop  down  menu list for picking  a  school  year level
```

```
        self.year_combo  = QComboBox()
        self.year_combo.addItems  (["1st Year", "2nd Year", "3rd Year", "4th Year", "5th Year"])


        # input  line box  for  typing  a course name and  section
        self.course_section  = QLineEdit()
        self.course_section.setPlaceholderText  ("Input your section here e.g. CPE21S4")


        # drop down menu  list  for  choosing program
        self.program  = QComboBox()
        self.program.addItems (["BS Computer Engineering", "BS Electronics Engineering", "BS
Electrical Engineering"])


        # choosing a date that  starts on  the  current calendar day
        self.date_needed  = QDateEdit(QDate.currentDate())
        self.date_needed.setCalendarPopup (True)
        # aligns  the  date text  cleanly  to the  left edge of the entry  box
        self.date_needed.setAlignment  (Qt.AlignmentFlag.AlignLeft)


        # drop  down menu list for selecting an approving laboratory  instructor
        self.professor   = QComboBox()
        self.professor.addItems (["Engr. Jimlord Quejado", "Engr. Menchie Rosales", "Engr. Ji Han
Gang"])


        # to  pair each description string  side by  side  with its matching input box
        self.form_layout.addRow ("Student ID:", self.student_id)
        self.form_layout.addRow("Student Name:", self.student_name)
        self.form_layout.addRow ("Year Level:", self.year_combo)
        self.form_layout.addRow("Course & Section:", self.course_section)
        self.form_layout.addRow ("Academic Program:", self.program)
        self.form_layout.addRow("Date Needed:", self.date_needed)
        self.form_layout.addRow  ("Assigned Professor:", self.professor)
        self.main_layout.addLayout (self.form_layout)


        self.tracking_label = QLabel("Your Active Borrowed Materials Tracking:")
        self.tracking_label.setObjectName ("sectionLabel")
        self.main_layout.addWidget (self.tracking_label)
```

```
        self.tracking_table    =  QTableWidget( 0, 5)
        self.tracking_table.setHorizontalHeaderLabels (["Request ID", "Items", "Status", "Lab
Ticket ID", "Update"])
        self.tracking_table.setMinimumHeight( 130)
        self.main_layout.addWidget (self.tracking_table)


        # creates a section text  label for the  active announcements message box area
        self.announcements_label  = QLabel ("Active Bulletins & Professor Announcements:")
        self.announcements_label.setObjectName  ("sectionLabel")
        self.main_layout.addWidget (self.announcements_label)


        # list widget box to render announcement text  notes
        self.announcements_list  = QListWidget()
        self.announcements_list.setMinimumHeight( 80)
        self.main_layout.addWidget (self.announcements_list)


        # search  box to check  the availability  of a material
        self.stock_search_label  =  QLabel ("Search Material Availability:")
        self.stock_search_label.setObjectName ("sectionLabel")
        self.main_layout.addWidget(self.stock_search_label)
        self.stock_search_input  = QLineEdit()
        self.stock_search_input.setPlaceholderText ("Search a material here e.g. Resistor")
        self.main_layout.addWidget (self.stock_search_input)
        self.stock_results     =  QListWidget()
        self.stock_results.setMinimumHeight( 80)
        self.main_layout.addWidget (self.stock_results)


        # horizontal material entry  row
        self.material_label = QLabel ("Add Materials to Request:")
        self.material_label.setObjectName("sectionLabel")


        # inserts the completed double column form block  into the  vertical main  layout
        self.main_layout.addWidget (self.material_label)
        # horizontal row  layout  to  line up item addition tools from  left to right
        self.material_entry_layout  =  QHBoxLayout()
        # text entry  box  for typing equipment and component names
```

```
        self.material_name_input = QLineEdit()
        self.material_name_input.setPlaceholderText("Material Name (e.g. Transistor, Resistor,
etc.)")


        # creates  a number  box  with up and down arrow adjusters for the quantity
        self.quantity_spin = ArrowSpinBox()
        self.quantity_spin.setRange( 1, 99)
        self.quantity_spin.setValue(1)


        # buttons to click add  items into the  display list
        self.add_item_btn  =  QPushButton ("Add Item")
        self.add_item_btn.setObjectName ("addItemBtn")


        # arranges  the  text field, quantity number box, and add button in a single row
        self.material_entry_layout.addWidget (self.material_name_input, 4)
        self.material_entry_layout.addWidget  (QLabel("Quantity:"))
        self.material_entry_layout.addWidget (self.quantity_spin, 1)
        self.material_entry_layout.addWidget (self.add_item_btn, 1)
        self.main_layout.addLayout  (self.material_entry_layout)


        # section  title label and applies a  custom identifier  name to  it
        self.section_label  = QLabel ("Materials that have been added")
        self.section_label.setObjectName ("boldSectionLabel")


        # adds the item  section label  and  the horizontal  tool  row to the  page
        self.main_layout.addWidget (self.section_label)


        # table grid with zero starting rows and two  labeled columns
        self.items_table = QTableWidget ( 0, 2)
        self.items_table.setHorizontalHeaderLabels (["Material Name", "Quantity"])
        self.items_table.setMinimumHeight (120)


        # puts  the grid  view table right  under the horizontal material entry  line
        self.main_layout.addWidget (self.items_table)


        # horizontal  row layout  for  the  action buttons at the bottom
```

```
        self.actions_layout  =  QHBoxLayout()
        # button to clear out  all  rows from the  table display
        self.clear_items_btn  =  QPushButton("Clear Items")
        self.clear_items_btn.setObjectName ("clearItemsBtn")

        # button to  submit the  final borrowing list to  the instructor
        self.submit_request_btn = QPushButton  ("Submit Borrowing Request")
        self.submit_request_btn.setObjectName("submitRequestBtn")

        # adds an  empty space that pushes the  clear button to the left and submit  to the right.
        self.actions_layout.addWidget (self.clear_items_btn)
        self.actions_layout.addStretch()
        self.actions_layout.addWidget  (self.submit_request_btn)
        self.main_layout.addLayout   (self.actions_layout)
```

A.5 Prof.py

```
import os
import sys
from pathlib import Path
from datetime import datetime
from wsgiref import headers
from Data_Model import Announcement
from CSVFileReaderWriter import CSVFileReaderWriter
from PyQt6.QtWidgets import QWidget, QMessageBox, QTableWidgetItem
from Prof_UI import ProfessorDashboardApp


class ProfessorDashboard(QWidget):
    def __init__(self, shared_requests_list=None, shared_announcement=None):
        super().__init__()

        # saves  the student requests list  if  it exists, or makes  a new empty list if  it is
missing
        self.shared_requests = shared_requests_list if shared_requests_list is not None else []
        # saves  the professor announcements list  if  it exists, or makes a new empty list if  it
is  missing
        self.shared_announcements = shared_announcement if shared_announcement is not None else []
```

```
        # creates  an instance of  the  interface layout
        self.ui = ProfessorDashboardApp()
        self.ui.setupUi(self)

        # tracking file path
        self.csv_handler = CSVFileReaderWriter()
        self.csv_filepath   = "ITSO_Maintenance_data.csv"

        # signals for approve  and decline  button
        self.ui.approve_btn.clicked.connect(self.process_approval)
        self.ui.reject_btn.clicked.connect(self.process_rejection)

        # signal  for the  announcement publish button
        self.ui.publish_btn.clicked.connect(self.publish_announcement)

        # automatically  sizes the  table  columns to fit the text inside them
        self.ui.requests_table.resizeColumnsToContents()
        # locks the column widths permanently  so they  cannot  be stretched  or  messed  up to
look neatly

self.ui.requests_table.horizontalHeader().setSectionResizeMode(self.ui.requests_table.horizontalHea
der().ResizeMode.Fixed)

        # loads all student requests into the  table when the window opens
        self.load_requests_data()

    def publish_announcement(self):
        message = self.ui.announcement_input.text().strip()

        # stops  the operation and shows a  warning  popup if the announcement text is empty
        if not message:
            QMessageBox.warning(self, "Input Error", "Please write an announcement message first.")
            return

        # new announcement record  with  the  current date  and  time stamp
        current_time = datetime.now().strftime("%b %d, %I:%M %p")
```

```
        new_announcement = Announcement(author="Prof", message=message, time_posted=current_time)

        self.shared_announcements.insert(0, new_announcement)


        # displays a success alert window  and  clears out the announcement input line box

        QMessageBox.information(self, "Success", "Announcement has been published successfully to
all students!")

        self.ui.announcement_input.clear()


    def load_requests_data(self):

        # clears out the table grid rows before reloading clean  records

        self.ui.requests_table.setRowCount(0)


        # loops  through  the shared  requests list  array to build  the  grid table lines

        for req in self.shared_requests:

            row  = self.ui.requests_table.rowCount()

            self.ui.requests_table.insertRow(row)

            self.ui.requests_table.setItem(row, 0, QTableWidgetItem(req.student_id))

            self.ui.requests_table.setItem(row, 1, QTableWidgetItem(req.name))

            self.ui.requests_table.setItem(row, 2, QTableWidgetItem(req.course_section))

            self.ui.requests_table.setItem(row, 3, QTableWidgetItem(str(req.date_needed))) #
converts the date into plain text to prevent crashes

            self.ui.requests_table.setItem(row, 4, QTableWidgetItem(req.items))

            self.ui.requests_table.setItem(row, 5, QTableWidgetItem(req.status))

            self.ui.requests_table.setItem(row, 6, QTableWidgetItem(req.timestamp_submitted))


            # displays the  exact time the  request  form  arrived from the student

            time_text = getattr(req, 'timestamp_submitted', '')

            self.ui.requests_table.setItem(row, 6, QTableWidgetItem(time_text))


    def process_approval(self):

        row  = self.ui.requests_table.currentRow()

        # stops  the  operation with  a warning error alert  if  the  instructor clicks approval
without selecting a row

        if row < 0:

            QMessageBox.warning(self, "Selection Error", "Please select a request from the table
grid first.")

            return
```

```
        comment =  self.ui.comment_input.toPlainText().strip()
        self.shared_requests[row].status =  "Approved"


        self.save_to_csv()


        # shows  an  alert card popup  with details  regarding the approval step
        QMessageBox.information(
            self, "Action Verified",
            f"Request Approved.\nComment note attached: {comment if comment else 'None'}"
        )
        self.clear_workspace()


    def process_rejection(self):
        row  = self.ui.requests_table.currentRow()
        if row < 0:
            QMessageBox.warning(self, "Selection Error", "Please select a request from the table
grid first.")
            return


        comment = self.ui.comment_input.toPlainText().strip()
        # stops the operation and triggers  an  alert if the feedback  text area is blank
        if not comment:
            QMessageBox.warning(
                self, "Validation Error", "You must provide a feedback comment explanation reason
before executing a Rejection action step.")
            return


        self.shared_requests[row].status   = "Rejected"
        self.save_to_csv()
        # shows a popup warning window containing the typed decline reasons string note
        QMessageBox.warning(
            self, "Action Verified",
            f"Request Declined.\nReason: {comment}"
        )
        self.clear_workspace()
    def save_to_csv(self):
```

```
        containers  =  ["Request ID", "Student ID", "Name", "Course Section", "Date Needed",
"Items", "Status", "Lab ID", "Time Submitted", "Time Returned"]
        if os.path.exists(self.csv_filepath):
            os.remove(self.csv_filepath)
        self.csv_handler.write(self.csv_filepath, containers)
        for req in self.shared_requests:
            record_row  = [
                str(req.request_id), req.student_id, req.name, req.course_section,
                str(req.date_needed), req.items, req.status, req.lab_id,
                req.timestamp_submitted, req.timestamp_returned
            ]
            self.csv_handler.write(self.csv_filepath, record_row)


    def clear_workspace(self):
        # resets the  layout  text fields and clears  any  selected rows  inside  the table grid
        self.ui.comment_input.clear()
        self.ui.requests_table.clearSelection()
        self.load_requests_data()
```

A.6 Prof_UI.py

```
from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel,
QTextEdit, QPushButton, QTableWidget, QLineEdit)


class ProfessorDashboardApp(QWidget):
    def setupUi( self, widget):


        # to identify the layout container box  name  so  it matches style sheet rules
        widget.setObjectName ("ProfessorDashboard")


        # layout  that  stacks widgets straight  down from top to bottom
        main_layout = QVBoxLayout (widget)
        main_layout.setContentsMargins (15, 15, 15, 15)
        main_layout.setSpacing (15)


        # main display title  text  label
        title =  QLabel("Professor Dashboard")
```

```
        title.setObjectName ("pageTitle")


        # smaller secondary explanation  text line  label
        subtitle = QLabel("Review student requests, log decision parameters, and update status.")
        subtitle.setObjectName ("pageSubtitle")


        # adds  both header  elements  down into the vertical layout stack
        main_layout.addWidget (title)
        main_layout.addWidget(subtitle)


        # creates  a structural  section  descriptor text label  widget
        table_label = QLabel("Incoming Material Requests  Awaiting Review:")
        table_label.setObjectName ("sectionLabel")
        main_layout.addWidget(table_label)


        # creates a  grid  view presentation  table with zero  rows  and  six columns
        self.requests_table = QTableWidget( 0 , 7)
        self.requests_table.setHorizontalHeaderLabels([
            "Request Student ID", "Student Name", "Course & Section", "Date Needed", "Requested
Items", "Status", "Time Submitted"
        ])
        self.requests_table.setMinimumHeight (220)
        main_layout.addWidget (self.requests_table)


        #announcement feature
        # text label header  for the  announcement writing  section
        announcement_label = QLabel("Publish New Announcement to Students:")
        announcement_label.setObjectName("sectionLabel")
        main_layout.addWidget (announcement_label)


        # text box row layout to align  the input  field and publish button  horizontally
        announcement_row =  QHBoxLayout()


        # text entry box  for typing out lab  announcement text
        self.announcement_input  = QLineEdit()
        self.announcement_input.setPlaceholderText ("e.g. Lab items must be returned before 4:00 PM
today.")
```

```
        # clickable  action button to publish the announcement  text
        self.publish_btn   = QPushButton ("Publish")
        self.publish_btn.setObjectName("publishBtn")

        # arranges the announcement box  and send button side by side in the  row
        announcement_row.addWidget(self.announcement_input, 4)
        announcement_row.addWidget (self.publish_btn, 1)
        main_layout.addLayout(announcement_row)

        # text label header  for the  decision feedback section
        comment_label   =  QLabel ("Decision Comment / Rejection Reason (Required if Rejecting):")
        comment_label.setObjectName("sectionLabel")
        main_layout.addWidget (comment_label)

        # multi line comment box with helpful instructional hint text
        self.comment_input  = QTextEdit()
        self.comment_input.setPlaceholderText ("Enter feedback note here, e.g., 'Approved. Return
before 5:00 PM.'")
        self.comment_input.setMaximumHeight (80)
        main_layout.addWidget(self.comment_input)

        # horizontal  strip  layout  row  for  the main action buttons
        actions_layout  = QHBoxLayout()

        # decline  button  action  trigger component
        self.reject_btn    = QPushButton("Decline Request")
        self.reject_btn.setObjectName ("rejectBtn")

        # approval button action trigger  component
        self.approve_btn =  QPushButton ("Approve Request")
        self.approve_btn.setObjectName("approveBtn")

        # adds an empty space that  pushes the reject button to the left and approve  to  the right
        actions_layout.addWidget (self.reject_btn)
        actions_layout.addStretch()
```

```
        actions_layout.addWidget(self.approve_btn)


        # inserts the  completed button  strip  row  down  into  the page layout
        main_layout.addLayout (actions_layout)
```

A.7 Staff.py

```
from PyQt6.QtWidgets import QWidget, QMessageBox, QTableWidgetItem, QPushButton, QInputDialog
import os
from CSVFileReaderWriter import CSVFileReaderWriter
from InventoryManager import InventoryManager
from Staff_UI import StaffDashboardApp
class StaffDashboard(QWidget):
    def __init__(self, shared_requests_list=None, inventory_manager=None):
        super().__init__()
        # saves  the student requests list if it  exists, or  makes a  new empty list  if it is
missing
        self.shared_requests  =  shared_requests_list if shared_requests_list is not None else []
        # inventory  that is saved in inventory_data.csv (shared  with the student  search box)
        self.inventory = inventory_manager if inventory_manager is not None else InventoryManager()


        self.ui = StaffDashboardApp()
        self.ui.setupUi(self)


        # to  fix the  sizes so the text  on the  column  would fit in the inventory table
        self.ui.inventory_table.resizeColumnsToContents()
        self.ui.inventory_table.horizontalHeader().setStretchLastSection(True)


        # fix sizes for  the student  requests  table
        self.ui.orders_table.resizeColumnsToContents()
        # signal  button for  stock function
        self.ui.add_stock_btn.clicked.connect(self.add_inventory_stock)


        # fill up the tables  with data  when the program opens
        self.display_inventory()
        self.display_student_requests()
```

```
    # whole function

    def display_inventory(self):

        # shows the live inventory (the same list that is saved  in  inventory_data.csv)

        self.ui.inventory_table.setRowCount(0)

        for stock in self.inventory.items:

            row   =   self.ui.inventory_table.rowCount()

            self.ui.inventory_table.insertRow(row)

            self.ui.inventory_table.setItem(row, 0, QTableWidgetItem(stock.material_name))

            self.ui.inventory_table.setItem(row, 1,
QTableWidgetItem(str(stock.quantity_available)))

        self.ui.inventory_table.resizeColumnsToContents()



    def add_inventory_stock(self):

        item_name = self.ui.item_name_input.text().strip()

        quantity  =   self.ui.item_qty_spin.value()  # (now a number instead of text)


        if not item_name:

            QMessageBox.warning(self, "Input Error", "Please provide a valid material component
name identifier.")

            return


        # adds to the existing  item  or creates a new one, then saves inventory_data.csv

        try:

            self.inventory.add_stock(item_name, quantity)

        except Exception as e:

            QMessageBox.warning(self, "Save Error", f"Stock was added but Inventory_data.csv could
not be saved (is it open in another program?):\n{e}")

        self.display_inventory()

        self.ui.item_name_input.clear()

        self.ui.item_qty_spin.setValue(1)


    def display_student_requests(self):

        # gets the  list  of student requests and  shows it on  the  screen

        self.ui.orders_table.setRowCount(0)


        for idx, req in enumerate(self.shared_requests):

            row = self.ui.orders_table.rowCount()
```

```
            self.ui.orders_table.insertRow(row)

            self.ui.orders_table.setItem(row, 0, QTableWidgetItem(req.student_id))

            self.ui.orders_table.setItem(row, 1, QTableWidgetItem(req.name))

            self.ui.orders_table.setItem(row, 2, QTableWidgetItem(req.items))

            self.ui.orders_table.setItem(row, 3, QTableWidgetItem(req.status))

            self.ui.orders_table.setItem(row, 4, QTableWidgetItem(req.lab_id if req.lab_id else
"Not Assigned"))


            # trigger  action  button

            release_btn = QPushButton("Release Items")

            release_btn.setObjectName("releaseBtn")


            # check the status to see  if the button should be turned off  if the request  is  not
approved  yet

            if req.status ==  "Approved":

                release_btn.setEnabled(True)

            else:

                release_btn.setEnabled(False)

                if req.status in ["Released by Laboratory Staff", "Pending Return Confirmation",
"Returned / Completed"]:

                    release_btn.setText("Released")

                else:

                    release_btn.setText("Unavailable")


            # saves the current  row index  inside  the button so  we know  which row was  clicked

            release_btn.setProperty("row_index", idx)

            # buttons  and signals

            release_btn.clicked.connect(self.handle_release_click)

            self.ui.orders_table.setCellWidget(row, 5, release_btn)


            confirm_return_btn = QPushButton("Confirm Return")

            confirm_return_btn.setObjectName("addStockBtn")



            if req.status == "Pending Return Confirmation":

                confirm_return_btn.setEnabled(True)

            else:
```

```
                confirm_return_btn.setEnabled(False)
                if req.status == "Returned / Completed":
                    confirm_return_btn.setText("Cleared")
                else:
                    confirm_return_btn.setText("Awaiting Return")

            confirm_return_btn.setProperty("row_index", idx)
            confirm_return_btn.clicked.connect(self.handle_release_return_click)
            self.ui.orders_table.setCellWidget(row, 6, confirm_return_btn)

        # to refresh the column  widths after  loading  text  rows
        self.ui.orders_table.resizeColumnsToContents()

    def release_items(self, target_request_idx):
        # changes the request status to  show  that items were given to the  student
        req = self.shared_requests[target_request_idx]

        # makes sure every  requested  item exists in  the inventory and has  enough units  before releasing
        allocations, problems = self.inventory.check_request(req.items)
        if problems:
            QMessageBox.warning(self, "Cannot Release", "This request cannot be released:\n\n" + "\n".join(problems))
            return

        lab_id, confirm   =  QInputDialog.getText(self, "Lab Ticket ID", f"Enter the Lab Ticket ID for {req.name}'s request:")
        if not confirm or not lab_id.strip():
            QMessageBox.warning(self, "Action Cancelled", "A valid Lab ID Ticket Number assignment is required to release physical inventory items.")
            return
        req.lab_id = lab_id.strip()
        req.status = "Released by Laboratory Staff"
        self.save_to_csv()  # save the updated request status to the CSV file

        # takes the borrowed units out of the available  stock and saves inventory_data.csv
        try:
            self.inventory.borrow(allocations)
```

```
        except Exception as e:
            QMessageBox.warning(self, "Save Error", f"Inventory_data.csv could not be saved (is it open in another program?):\n{e}")
        self.display_inventory()
        QMessageBox.information(self, "Inventory Released", f"Materials safely assigned and handed out to {req.name}.")
        self.display_student_requests()


    def handle_release_click(self):
        button  = self.sender()
        if button:
            target_row = button.property("row_index")
            self.release_items(target_row)


    def handle_release_return_click(self):
        # identifies  which button was clicked and reads its  row  number
        button =  self.sender()
        if button:
            target_row =  button.property("row_index")
            req = self.shared_requests[target_row]


            req.status = "Returned / Completed"
            self.save_to_csv()


            # puts the returned units back into  the available stock and  saves  inventory_data.csv
            try:
                self.inventory.return_items(req.items)
            except Exception as e:
                QMessageBox.warning(self, "Save Error", f"Inventory_data.csv could not be saved (is it open in another program?):\n{e}")
            self.display_inventory()
            QMessageBox.information(self, "Return Completed", f"Ticket [{req.lab_id}] cleared! Materials successfully verified and returned to storage shelves layout.")
            self.display_student_requests()


    def save_to_csv(self):
        csv_filepath = "ITSO_Maintenance_data.csv"
        csv_handler = CSVFileReaderWriter()
```

```
        containers = ["Request ID", "Student ID", "Name", "Course Section", "Date Needed", "Items",
"Status", "Lab ID", "Time Submitted", "Time Returned"]

        if os.path.exists(csv_filepath):

            os.remove(csv_filepath)

        csv_handler.write(csv_filepath, containers)

        for req in self.shared_requests:

            record_row  = [

                str(req.request_id), req.student_id, req.name, req.course_section,

                str(req.date_needed), req.items, req.status, req.lab_id,

                req.timestamp_submitted, req.timestamp_returned

            ]

            csv_handler.write(csv_filepath, record_row)
```

A.8 Staff_UI.py

```
from PyQt6.QtWidgets import  QWidget, QVBoxLayout, QHBoxLayout, QLabel, QLineEdit, QSpinBox,
QPushButton, QTableWidget

from arrowspinbox import  ArrowSpinBox


class StaffDashboardApp (QWidget):

    def setupUi( self, widget):

        widget.setObjectName ("StaffDashboard")


        self.main_layout  = QVBoxLayout(widget)

        self.main_layout.setContentsMargins (15 , 15, 15 , 15)

        self.main_layout.setSpacing ( 12)


        self.title = QLabel ("Laboratory Stockroom & Staff Dashboard")

        self.title.setObjectName ("pageTitle")


        self.subtitle = QLabel ("Manage physical hardware count tracking and release verified
inventory.")

        self.subtitle.setObjectName ("pageSubtitle")

        self.main_layout.addWidget (self.title)

        self.main_layout.addWidget (self.subtitle)


        # inventory adjustment  for updating  stock row controls

        self.inv_label = QLabel  ("Item Stock Procurement Adjuster:")
```

```
        self.inv_label.setObjectName ("sectionLabel")
        self.main_layout.addWidget (self.inv_label)


        self.entry_layout  =  QHBoxLayout()
        self.item_name_input  = QLineEdit()
        self.item_name_input.setPlaceholderText ("Component Name (e.g. Oscilloscope, Arduino)")


        self.item_qty_spin  =  ArrowSpinBox()
        self.item_qty_spin.setRange ( 1, 999)


        self.add_stock_btn  = QPushButton ("Update / Add Stock")
        self.add_stock_btn.setObjectName ("addStockBtn")


        self.entry_layout.addWidget (self.item_name_input, 4)
        self.entry_layout.addWidget (QLabel("Qty:"))
        self.entry_layout.addWidget(self.item_qty_spin, 1)
        self.entry_layout.addWidget  (self.add_stock_btn, 1)
        self.main_layout.addLayout(self.entry_layout)


        # table of showing  the  current items  in the stock
        self.stock_label = QLabel ("Current On-Hand Stock Inventory Ledger:")
        self.stock_label.setObjectName ("sectionLabel")
        self.main_layout.addWidget(self.stock_label)


        self.inventory_table = QTableWidget( 0, 2)
        self.inventory_table.setHorizontalHeaderLabels (["Material/Hardware Model",  "Units
Available"])
        self.inventory_table.setMinimumHeight ( 150)
        self.main_layout.addWidget(self.inventory_table)


        # label for  student requests section
        self.student_req = QLabel ("Verified Request Release:")
        self.student_req.setObjectName ("sectionLabel")
        self.main_layout.addWidget  (self.student_req)


        # table showing active student requests and their  tracking  status
```

```
        self.orders_table = QTableWidget ( 0, 7)

        self.orders_table.setHorizontalHeaderLabels (["Student ID", "Borrower", "Requested Items",
"Approval Status", "Lab Ticket ID", "Released", "Returned"])

        self.orders_table.setMinimumHeight ( 180)

        self.main_layout.addWidget (self.orders_table)
```

A.9 ITSO.py

```
import os

# importing  the shutil to handle file operations like  copying  or deleting

# complete  folder  trees instead of  rmdir  because of limited  usage and  restrictions  when
deleting a data

import shutil

from PyQt6.QtWidgets import QWidget, QTableWidgetItem, QMessageBox, QFileDialog

from ITSO_UI import ITSODashboardApp

from CSVFileReaderWriter import CSVFileReaderWriter


class ITSODashboard(QWidget):

    def __init__(self):

        super().__init__()

        self.ui  =   ITSODashboardApp()

        self.ui.setupUi(self)

        self.csv_filepath = "ITSO_Maintenance_data.csv"

        self.csv_handler = CSVFileReaderWriter()


        self.ui.refresh_btn.clicked.connect(self.sync_csv_data)

        self.ui.gen_folder_btn.clicked.connect(self.generate_backup_folder)

        self.ui.del_folder_btn.clicked.connect(self.delete_backup_folder)


        # automatically  sizes  the  table columns to fit the text inside them

        self.ui.log_table.resizeColumnsToContents()


self.ui.log_table.horizontalHeader().setSectionResizeMode(self.ui.log_table.horizontalHeader().Resi
zeMode.Interactive)

        self.sync_csv_data()


    def sync_csv_data(self):

        self.ui.log_table.setRowCount(0)
```

```
        # checks  if the log  file  exists before trying to  read it

        if not os.path.exists(self.csv_filepath):

            # stops the function  here if  the  file is missing

            QMessageBox.warning(self, "File Not Found", f"The tracking data file
'{self.csv_filepath}' could not be located in the project folder.\n\nPlease ensure the file exists
before syncing.")

            return

        try:

            self.new_data = self.csv_handler.read(self.csv_filepath)


            # checks if the loaded  data  matches  the  title header  list

            # and adding index [0] to check if the first row  matches to the 10 list  of columns

            if self.new_data and str(self.new_data[0][0]).strip() == "Request ID":

                # skips  the  first  row so  it does not add column titles to the  table cells

                rows_to_turn = self.new_data[1:] # runs only if the file matches, it cuts off the
very first row (the column with titles) and passes the rest of the student data lines down to be
loaded into the table grid slots

            # runs  if the file does  not have a  title line at the top

            else:

                # keeps the  full list  because  no  header lines  need to be skipped

                rows_to_turn = self.new_data


            # loops through  each  separate  row line in  the data  file list

            for row_data in rows_to_turn:

                # discards blank  rows  to avoid empty  spaces on the screen

                if not row_data or len(row_data) == 0:

                    continue # skips this line completely and moves to the next one

                row_idx   = self.ui.log_table.rowCount()

                self.ui.log_table.insertRow(row_idx)

                for col_idx, value in enumerate(row_data):

                    # sets  all columns  limit  to  fit in the table width

                    if col_idx < 10: # access 3 more slots, allowing the program to read and
display all 10 tracking columns

                        display_text =  str(value)


                        # changes data  to  normal  text and puts it directly into table cells

                        self.ui.log_table.setItem(row_idx, col_idx,
QTableWidgetItem(str(display_text)))

                # refreshes the  column width calculations after loading the new texts
```

```
                self.ui.log_table.resizeColumnsToContents() # adjusts the column boxes so that long
timestamp strings fit and looks neat


        except Exception as e:
            QMessageBox.critical(self, "File Error", f"Failed to load the file data:\n{e}")


    def generate_backup_folder(self):
        try:
            # opens a popup window  so the  user can select  a  folder on the computer
            path  = QFileDialog.getExistingDirectory(self, "Select or Create Location for Backup",
"")
            # makes  sure the  user actually clicked a folder instead of  closing the popup
            if path:
                backup_dir = os.path.join(path, "ITSO Backup Folder")
                # checks  if the  backup folder  does not exist on the computer  yet
                if not os.path.exists(backup_dir):
                    # creates a  brand new backup  folder on the  hard drive path
                    os.mkdir(backup_dir)


                # verifies if the current data  log file is  inside  the  project  folder
                if os.path.exists(self.csv_filepath):
                    destination_file =  os.path.join(backup_dir, "backup_log.csv")
                    # uses  shutil.copy to safely  clone the  live  csv file over into  the backup
folder location
                    shutil.copy(self.csv_filepath, destination_file)
                    QMessageBox.information(self, "Success", f"Backup folder has created and file
saved at:\n{destination_file}")
                # runs this warning path  if the  log file  cannot  be found by python
                else:
                    QMessageBox.warning(self, "Info", "Backup folder created, but no active student
CSV log was found to copy.")
        except Exception as e:
            QMessageBox.warning(self, "Error", f"Cannot create backup system folder:\n{e}")


    def delete_backup_folder(self):
        try:
            # opens  a popup window so  the user can select  which folder to erase
            folder  = QFileDialog.getExistingDirectory(self, "Select ITSO_Backups Folder to
Delete", "")
```

```
            # ensures  a folder was chosen and checks  that it actually exists on  disk
            if folder and os.path.exists(folder):
                # uses  shutil.rmtree  to force delete the selected backup  folder and all contents
inside  it
                shutil.rmtree(folder)
                QMessageBox.information(self, "Success", "Backup directory and all its logs removed
successfully.")
            # activates if  the folder  path  selection  was  left empty  or  cancelled
            else:
                QMessageBox.warning(self, "Error", "No folder was selected or the specific
location does not exist.")
        except Exception as e:
            QMessageBox.warning(self, "Error", f"Folder deletion process failed:\n{e}")
```

A.10 ITSO_UI.py

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton, QTableWidget


class ITSODashboardApp(QWidget):
    def setupUi(self, widget):


        widget.setObjectName("ITSODashboard")


        self.main_layout = QVBoxLayout(widget)
        self.main_layout.setContentsMargins(15, 15, 15, 15)
        self.main_layout.setSpacing(12)


        self.title = QLabel("ITSO Maintenance Dashboard")
        self.title.setObjectName("pageTitle")


        self.subtitle = QLabel("Checks student request files that are saved automatically in the
table.")
        self.subtitle.setObjectName("pageSubtitle")
        self.main_layout.addWidget(self.title)
        self.main_layout.addWidget(self.subtitle)


        self.controls_layout = QHBoxLayout()
        self.table_label = QLabel("Synchronized ITSO Network Device Records:")
```

```
        self.table_label.setObjectName("sectionLabel")


        self.refresh_btn  = QPushButton("Sync / Refresh CSV Log")
        self.refresh_btn.setObjectName("refreshBtn")
        self.gen_folder_btn = QPushButton("Generate Backup Folder")
        self.gen_folder_btn.setObjectName("refreshBtn")
        self.del_folder_btn = QPushButton("Delete Backup Folder")
        self.del_folder_btn.setObjectName("refreshBtn")


        self.controls_layout.addWidget(self.table_label)


        self.controls_layout.addStretch()
        self.controls_layout.addWidget(self.refresh_btn)
        self.controls_layout.addWidget(self.gen_folder_btn)
        self.controls_layout.addWidget(self.del_folder_btn)
        self.main_layout.addLayout(self.controls_layout)


        self.log_table = QTableWidget(0, 10)
        self.log_table.setHorizontalHeaderLabels([
            "Request ID", "Student ID", "Student Name", "Course & Sec", "Date Needed", "Requested
Items", "System Status", "Lab ID", "Time Submitted", "Time Returned"
        ])
        self.log_table.setMinimumHeight(350)
        self.main_layout.addWidget(self.log_table)
```

A.11 InventoryManager.py

```
# whole file: keeps the stock list and  saves it to inventory_data.csv
import os
import re
from datetime import datetime
from CSVFileReaderWriter import CSVFileReaderWriter
from Data_Model import StockMaterials


# column titles of the  inventory csv  file
INVENTORY_HEADER = ["Material Name", "Total Stock", "Available", "Borrowed", "Last Updated"]
```

```
# starting  stock used only when the inventory csv does  not  exist  yet
DEFAULT_STOCK = [
    ("Resistors Combo Assortment", 250),
    ("NPN Transistors 2N2222", 85),
    ("Digital Multimeter Fluke", 14),
]



class InventoryManager:
    # keeps the  stockroom  list in memory and  mirrors every change into  inventory_data.csv
    def __init__(self , filepath = "Inventory_data.csv"):
        self.filepath = filepath
        self.csv_handler =  CSVFileReaderWriter()
        # this  list is shared  with the  other dashboards, so it is only ever  changed in place
        self.items = []
        self.load()

    # file handling
    def load(self):
        loaded = []
        if os.path.exists (self.filepath):
            try:
                rows = self.csv_handler.read (self.filepath)
                for row in rows:
                    if not row or len(row) < 3 or str(row[0]).strip()  == "Material Name":
                        continue
                    loaded.append(StockMaterials(
                        item_name=str(row[0]).strip() ,
                        total=int(row[1]) ,
                        available=int(row[2]) ,
                        borrowed=int(row[3]) if len(row) > 3 and row[3] != "" else 0,
                        last_updated=row[4] if len(row) > 4 else ""))
            except Exception as e:
                print(f"Could not read inventory file, using default stock: {e}")
                loaded = []
```

```
        if not loaded:
            stamp =  self.now()
            loaded = [StockMaterials(n , q , last_updated = stamp) for n, q in DEFAULT_STOCK]

        self.items[:] = loaded
        # creates  the  file on first run  so it always exists next to the program
        if not os.path.exists (self.filepath):
            self.save()

    def save(self):
        # writes to a temporary file first, then swaps it in, so a crash  never  leaves a  half
written file
        temp_path = self.filepath +  ".tmp"
        if os.path.exists (temp_path):
            os.remove (temp_path)
        self.csv_handler.write (temp_path , INVENTORY_HEADER)
        for s in self.items:
            self.csv_handler.write(temp_path, [
                s.material_name, str(s.total_stock), str(s.quantity_available),
                str(s.quantity_borrowed), s.last_updated])
        os.replace( temp_path , self.filepath)

    @staticmethod
    def now():
        return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # staff adds stock
    def add_stock(self, name, qty):
        # adds  to  an existing item (same name, any  capitalisation) or  creates a brand new one
        stamp = self.now()
        for s in self.items:
            if s.material_name.lower() == name.lower():
                s.total_stock += qty
                s.quantity_available +=  qty
                s.last_updated  =  stamp
                self.save()
```

```
                return s
        new_item = StockMaterials(name , qty , last_updated  =  stamp)
        self.items.append(new_item)
        self.save()
        return new_item


    # borrowing / returning
    @staticmethod
    def parse_items(items_text):
        # turns "resistor (x3), transistor (x4)" into [("resistor", 3), ("transistor", 4)]
        return [(n.strip(), int(q)) for n, q in re.findall(r"([^,]+?)\s*\(x(\d+)\)", items_text)]
    def find_item(self, requested_name):
        # exact  name  first, then a  name  that contains / is contained  in what the  student
typed
        key = requested_name.strip().lower()
        for s in self.items:
            if s.material_name.lower() == key:
                return s
        matches = [s for s in self.items
                   if key in s.material_name.lower() or s.material_name.lower() in key]
        return matches[0] if len (matches) == 1  else None


    def resolve_request(self, items_text):
        # matches  every  requested  line  to  a  stock item; returns ([(stock, qty)], [problem
messages])
        wanted = {}
        problems  = []
        parsed =  self.parse_items(items_text)
        if not parsed:
            problems.append ("The request items could not be read.")
        for name, qty in parsed:
            stock = self.find_item (name)
            if stock is None:
                problems.append (f"'{name}' is not in the inventory (or matches more than one
item).")
            else:
                entry = wanted.setdefault(id(stock), [stock, 0])
```

```
                entry[1]  += qty
        return [tuple(e) for e in wanted.values()] , problems


    def check_request( self, items_text):
        # same as resolve_request but  also reports items that do not have  enough units left
        allocations , problems = self.resolve_request (items_text)
        for stock , qty in allocations:
            if qty > stock.quantity_available:
                problems.append(f"{stock.material_name}: needs {qty} , only
{stock.quantity_available} available.")
        return allocations , problems


    def borrow( self, allocations):
        # takes  the  borrowed  units  out  of  available
        stamp   = self.now()
        for stock, qty in allocations:
            stock.quantity_available -= qty
            stock.quantity_borrowed +=  qty
            stock.last_updated =  stamp
        self.save()


    def return_items( self , items_text):
        # puts  returned  units  back  into  available (never more  than  what is  recorded  as
borrowed)
        allocations, problems = self.resolve_request (items_text)
        stamp = self.now()
        for stock, qty in allocations:
            back = min( qty , stock.quantity_borrowed)
            stock.quantity_available += back
            stock.quantity_borrowed -= back
            stock.last_updated = stamp
        self.save()
        return problems
```

A.12 Data_Model.py

```
from datetime import date
```

```
class BorrowRequestData:

    def __init__(self, request_id: int, student_id: str, name: str, course_section: str,
date_needed: date, items: str):

        # to keep track  of  all data  each  inside  a single borrowing request  object

        self.request_id = request_id

        self.student_id = student_id

        self.name = name

        self.course_section = course_section

        self.date_needed = date_needed

        self.items = items

        self.status   = "Pending Approval"

        # to  store  the laboratory tracking  number  assigned  when  staff releases  items

        self.lab_id = ""

        # records the exact date  and time when the student sends  the form

        self.timestamp_submitted   =  ""

        # records the exact date and time when the  materials  are brought  back

        self.timestamp_returned = ""


# moved here from main.py and extended  with total / borrowed / last updated

class StockMaterials:

    def __init__(self, item_name: str, total: int, available: int = None, borrowed: int = 0,
last_updated: str = ""):

        # one line of the stockroom  inventory

        self.material_name = item_name

        self.total_stock = int(total)

        # units currently on the  shelf (total minus the ones borrowed)

        self.quantity_available = int(total) if available is None else int(available)

        # units currently with  students

        self.quantity_borrowed = int(borrowed)

        self.last_updated = last_updated


class Announcement:

    def __init__(self, author: str, message: str, time_posted: str):

        # keeps  track of a single announcement  message published by  a  laboratory instuctor

        self.author = author

        self.message = message
```

```
        self.time_posted = time_posted
```

A.13 FileReader.py

```
# create  our parent class

class FileReaderWriter():

    # base  layout for  reading  files  that returns an empty  list  if  not  changed

    def read(self):

        print("This is the default read method")

    # returns an  empty  list as a safety  backup to prevent the  program from crashing if no  data
is  found

        return[]

    # base  layout for  writing data into files

    def write(self):

        print("This is the  default write method")
```

A.14 CSVFileReaderWriter.py

```
from FileReader import FileReaderWriter

import csv


    # this  file and function  is used for converting  the  borrowing  slip  of  the  student

    # from  the  student  dashboard  into csv  file and  received  to the itso dashboard


    # opens  a csv file  and  returns  all  lines as a list

class CSVFileReaderWriter(FileReaderWriter):

    def read(self, filepath):

        with open(filepath, newline='') as  csvfile:

            data = csv.reader(csvfile, delimiter=',', quotechar='|')

            return list(data)


    # opens  a csv  file and  adds a  new  line of  data to  the  end of  it

    def write(self, filepath, data):

    # opens the  file to  add new  student  lines without  deleting  the  old  ones

        with open(filepath, 'a', newline  = '') as csvfile:

            writer = csv.writer(csvfile,  delimiter=',', quotechar='|', quoting=csv.QUOTE_MINIMAL)

            writer.writerow(data)
```

A.15 arrowspinbox.py

```
from PyQt6.QtWidgets import QSpinBox, QStyle, QStyleOptionSpinBox
from PyQt6.QtGui import QPainter, QPolygonF, QColor
from PyQt6.QtCore import QPointF, Qt



class ArrowSpinBox(QSpinBox):
    # a  normal  spinbox that also draws its  own  up/down triangles  on the buttons
    def paintEvent(self, event):
        super().paintEvent(event)

        opt = QStyleOptionSpinBox()
        self.initStyleOption(opt)
        style = self.style()
        up_rect  = style.subControlRect(
            QStyle.ComplexControl.CC_SpinBox, opt, QStyle.SubControl.SC_SpinBoxUp, self)
        down_rect  =   style.subControlRect(
            QStyle.ComplexControl.CC_SpinBox, opt, QStyle.SubControl.SC_SpinBoxDown, self)

        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QColor("#000000") if self.isEnabled() else QColor("#888888"))

        self.draw_triangle(painter,  up_rect, pointing_up=True)
        self.draw_triangle(painter, down_rect, pointing_up=False)
        painter.end()

    def draw_triangle(self, painter, rect, pointing_up):
        cx = rect.center().x() + 0.5
        cy = rect.center().y()+0.5
        if pointing_up:
            points = [QPointF(cx - 4, cy + 2), QPointF(cx + 4, cy + 2), QPointF(cx, cy - 3)]
        else:
```

```
            points = [QPointF(cx - 4, cy - 2), QPointF(cx + 4, cy - 2), QPointF(cx, cy  +  3)]
        painter.drawPolygon(QPolygonF(points))
```

A.16 style.qss

```
/* Base */
QWidget {
    background-color: #a5a8ab;
    color: #000000;
    font-family: "Syne", "Outfit", "Segoe UI", sans-serif;
    font-size: 12px;
    font-weight: bold;
}

QLabel {
    background-color: transparent;
}

QMessageBox {
    background-color: #a5a8ab;
}
QMessageBox QLabel {
    color: #000000;
}
QMessageBox QPushButton {
    min-width: 70px;
}

/* Inputs (default = login screen: yellow fields) */
QLineEdit, QComboBox, QDateEdit, QSpinBox {
    background-color: #fabb0d;
    color: #000000;
    border: 1px solid #3a3a3a;
    border-radius: 8px;
    padding: 5px 10px;
    font-weight: bold;
```

```
}
QLineEdit:focus, QComboBox:focus, QDateEdit:focus, QSpinBox:focus {
    border: 2px solid #000000;
}
QLineEdit[error="true"] {
    border: 2px solid #e53935;
}

QComboBox QAbstractItemView {
    background-color: #ffffff;
    color: #000000;
    selection-background-color: #fabb0d;
    selection-color: #000000;
    border: 1px solid #fabb0d;
}

/* Show password checkbox */
QCheckBox {
    background-color: transparent;
    spacing: 8px;
}
QCheckBox::indicator {
    width: 22px;
    height: 22px;
    border: 1px solid #3a3a3a;
    border-radius: 5px;
    background-color: #fabb0d;
}
QCheckBox::indicator:checked {
    background-color: #000000;
    border: 2px solid #fabb0d;
}

/* Buttons (default = black with yellow text) */
QPushButton {
    background-color: #000000;
```

```
    color: #fabb0d;
    border: none;
    border-radius: 7px;
    padding: 6px 14px;
    font-weight: bold;
}
QPushButton:hover {
    background-color: #fabb0d;
    color: #000000;
}
QPushButton:pressed {
    background-color: #c99408;
    color: #000000;
}
QPushButton:disabled {
    background-color: #5c5f62;
    color: #c9c9c9;
}

/* Login screen */
QLabel#title {
    font-size: 24px;
    font-weight: bold;
    color: #000000;
}
QLabel#subtitle {
    font-size: 12px;
    color: #2b2b2b;
}
QPushButton#loginBtn {
    background-color: #000000;
    color: #fabb0d;
    padding: 8px 14px;
}
QPushButton#loginBtn:hover {
    background-color: #fabb0d;
```

```
    color: #000000;
}
QPushButton#clearBtn {
    background-color: #fabb0d;
    color: #000000;
    border: 1px solid #000000;
    padding: 8px 14px;
}
QPushButton#clearBtn:hover {
    background-color: #000000;
    color: #fabb0d;
}
QLabel#status {
    font-size: 11px;
}
QLabel#status[state="success"] {
    color: #14532d;
}
QLabel#status[state="error"] {
    color: #b71c1c;
}

/* Dashboard headings (yellow pills) */
QLabel#pageTitle {
    background-color: #fabb0d;
    color: #000000;
    border-radius: 22px;
    padding: 6px 22px;
    font-size: 26px;
    font-weight: normal;
}
QLabel#pageSubtitle {
    color: #1f1f1f;
    font-size: 11px;
    font-weight: normal;
    padding-left: 6px;
```

```
}
QLabel#sectionLabel,
QLabel#boldSectionLabel,
QLabel#formLabel {
    background-color: #fabb0d;
    color: #000000;
    border: 1px solid #3a3a3a;
    border-radius: 10px;
    padding: 3px 12px;
    font-weight: bold;
}

/* Dashboard input fields (white with yellow border) */
#StudentDashboard QLineEdit,
#StudentDashboard QComboBox,
#StudentDashboard QDateEdit,
#StudentDashboard QSpinBox,
#ProfessorDashboard QLineEdit,
#StaffDashboard QLineEdit,
#StaffDashboard QSpinBox {
    background-color: #ffffff;
    color: #000000;
    border: 1px solid #fabb0d;
    border-radius: 8px;
    padding: 4px 8px;
}
#StudentDashboard QLineEdit:focus,
#StudentDashboard QComboBox:focus,
#StudentDashboard QDateEdit:focus,
#StudentDashboard QSpinBox:focus,
#ProfessorDashboard QLineEdit:focus,
#StaffDashboard QLineEdit:focus,
#StaffDashboard QSpinBox:focus {
    border: 2px solid #fabb0d;
}
```

```
/* QSpinBox up/down buttons (explicit geometry so clicks work) */
QSpinBox {
    padding-right: 28px;
}
QSpinBox::up-button {
    subcontrol-origin: border;
    subcontrol-position: top right;
    width: 22px;
    border-left: 1px solid #fabb0d;
    border-bottom: 1px solid #fabb0d;
    border-top-right-radius: 8px;
    background-color: #f2f2f2;
}
QSpinBox::down-button {
    subcontrol-origin: border;
    subcontrol-position: bottom right;
    width: 22px;
    border-left: 1px solid #fabb0d;
    border-bottom-right-radius: 8px;
    background-color: #f2f2f2;
}
QSpinBox::up-button:hover, QSpinBox::down-button:hover {
    background-color: #fabb0d;
}
QSpinBox::up-button:pressed, QSpinBox::down-button:pressed {
    background-color: #d99f00;
}

QLineEdit::placeholder {
    color: #9a9a9a;
}

QTextEdit, QListWidget {
    background-color: #ffffff;
    color: #000000;
    border: 1px solid #fabb0d;
```

```
    border-radius: 8px;
    padding: 4px;
    font-weight: normal;
}
QListWidget::item:selected {
    background-color: #fabb0d;
    color: #000000;
}

/* Tables */
QTableWidget {
    background-color: #ffffff;
    color: #000000;
    border: 1px solid #fabb0d;
    border-radius: 6px;
    gridline-color: #fabb0d;
    font-weight: normal;
    selection-background-color: #ffe08a;
    selection-color: #000000;
}
QTableWidget::item {
    padding: 3px;
}
QHeaderView::section {
    background-color: #000000;
    color: #ffffff;
    border: 1px solid #fabb0d;
    padding: 4px 8px;
    font-weight: bold;
}
QTableCornerButton::section {
    background-color: #000000;
    border: 1px solid #fabb0d;
}

/* Scrollbars */
```

```
QScrollBar:vertical {
    background: #e8e8e8;
    width: 12px;
    border-radius: 6px;
    margin: 0;
}
QScrollBar::handle:vertical {
    background: #fabb0d;
    min-height: 24px;
    border-radius: 6px;
}
QScrollBar:horizontal {
    background: #e8e8e8;
    height: 12px;
    border-radius: 6px;
    margin: 0;
}
QScrollBar::handle:horizontal {
    background: #fabb0d;
    min-width: 24px;
    border-radius: 6px;
}
QScrollBar::add-line, QScrollBar::sub-line {
    width: 0;
    height: 0;
}

/* Dashboard action buttons */
QPushButton#addItemBtn,
QPushButton#clearItemsBtn,
QPushButton#submitRequestBtn,
QPushButton#publishBtn,
QPushButton#addStockBtn,
QPushButton#refreshBtn {
    background-color: #000000;
    color: #fabb0d;
```

```
}
QPushButton#addItemBtn:hover,
QPushButton#clearItemsBtn:hover,
QPushButton#submitRequestBtn:hover,
QPushButton#publishBtn:hover,
QPushButton#addStockBtn:hover,
QPushButton#refreshBtn:hover {
    background-color: #fabb0d;
    color: #000000;
}

/* Professor: decline / approve */
QPushButton#rejectBtn {
    background-color: #000000;
    color: #e53935;
}
QPushButton#rejectBtn:hover {
    background-color: #e53935;
    color: #ffffff;
}
QPushButton#approveBtn {
    background-color: #000000;
    color: #22c55e;
}
QPushButton#approveBtn:hover {
    background-color: #22c55e;
    color: #000000;
}

/* Staff: release button inside the table cells */
QPushButton#releaseBtn {
    background-color: #000000;
    color: #fabb0d;
    border-radius: 5px;
    padding: 3px 10px;
}
```

```
QPushButton#releaseBtn:hover {
    background-color: #fabb0d;
    color: #000000;
}
QPushButton#releaseBtn:disabled {
    background-color: #5c5f62;
    color: #d0d0d0;
}
```

A.17 app_icon.ico / app_icon.png / TIP_LOGO.png



A.18 Inventory_data.csv

```
Material Name,Total Stock,Available,Borrowed,Last Updated
Resistors Combo Assortment,250,250,0,2026-11-02 14:35:38
NPN Transistors 2N2222,85,85,0,2026-11-02 11:06:27
Digital Multimeter Fluke,14,14,0,2026-11-02 10:50:22
Breadboard,40,40,0,2026-11-02 11:14:48
Diode,6,6,0,2026-11-02 11:36:15
```

A.19 ITSO_maintenance_data.csv

```
Request ID,Student ID,Name,Course Section,Date Needed,Items,Status,Lab ID,Time Submitted,Time Returned

1,2516965,John Vince Acelajado,CPE21S4,2026-11-02,Resistor (x10),Approved,,2026-11-02 14:43:36,
```

Appendix B. User Guide

1. Extract the zip file and double-click the .exe file.
2. On the Login Window, choose your role (Student, Professor, Laboratory Staff, or ITSO Maintenance).
3. Type the username and password (Student / student1234, Professor / prof123, Staff / staff123, ITSO / itso123). Click “Show password” if you want to see what you typed.
4. Click Login. If it is wrong, an error shows and the window shakes.
5. Student: Fill in the form, search for a material, type the material name and quantity, then click Add Item. Click Submit Borrowing Request. You can follow the status in the tracking table.
6. Professor: Click a row in the table, write a comment, then click Approve Request or Decline Request. A comment is needed to decline. You can also post an announcement.
7. Laboratory Staff: Use Update / Add Stock to add materials. For an approved request, click Release Items and type the Lab Ticket ID. When the student returns the items, click Confirm Return.
8. ITSO**:** Click Sync / Refresh CSV Log to reload the records. Use Generate Backup Folder to copy the log, or Delete Backup Folder to remove a backup.
9. To leave, click Logout / Switch System Role or use the File menu.

Appendix C. Project Files and Links

Item	Link or File Name
Source code repository (e.g., GitHub)	https://github.com/RebagosBSCpE/OOP2-FINAL-PROJECT-GROUP3
Packaged application (.zip with .exe)	https://drive.google.com/file/d/113gFCSS5H0xnyuHubIvTLxRzDqh4TOov/view?usp=sharing
Presentation and Demo video (15 mins)	https://drive.google.com/file/d/1Ym7gbgIaM2W0_IetR_PFe7zkDkCPITfw/view?usp=sharing
Presentation slides	https://www.canva.com/design/DAHZw8x6UKc/fXf9yjSvyq016BJun6TC5w/edit

Appendix D. Work Distribution

Member	Tasks Completed	Files / Classes Written	Contribution (%)
Acelajado, John Vince D.	PPT Maker, Github Editor, FrontEnd, Styling, Packaging	Style.qss, arrowspinbox.py, app_icon.ico, app_icon.png, main.py, ITSO_UI.py, ITSO.py, LoginSystem.py, Prof_UI.py, Prof.py, Staff_UI.py, Staff.py, Student_UI.py, Student.py	100
Bal, Kevin Ranz J.	FrontEnd, BackEnd, Styling, Document Paper, Video Compiler	CSVFileReaderWrtier.py, FileReader.py, Data_Model.py, main.py, ITSO_UI.py, ITSO.py, LoginSystem.py, Prof_UI.py, Prof.py, Staff_UI.py, Staff.py, Student_UI.py, Student.py, style.qss	100

EVALUATION CRITERIA

Criteria	Description	Points
Problem and Objectives	Clear real-world problem; specific, measurable objectives	10
Scope and Output	Realistic scope; features, delimitations, and deliverables well defined	10
Functionality	All features in the scope work correctly, including input validation and file storage	20
Application of OOP Principles	Correct and meaningful use of encapsulation, abstraction, inheritance, polymorphism, and composition, clearly explained	25
GUI Design and Usability	Consistent styling, clear navigation, helpful feedback and dialogs	15
Results and Discussion	Complete screenshots and test cases; objectives evaluated with evidence	10
Documentation and Code Quality	Complete appendices, readable and commented code, working .exe	10
Total		100